



Rafael Ferreira Lucena

# **Sistema de Triagem de Doenças Foliares em Soja com Inteligência Artificial e Visão Computacional**

Niterói

2026



Rafael Ferreira Lucena

## **Sistema de Triagem de Doenças Foliares em Soja com Inteligência Artificial e Visão Computacional**

Trabalho de conclusão de curso apresentado ao Centro Universitário La Salle do Rio de Janeiro (UNILASALLE-RJ) como requisito parcial à obtenção de grau de Bacharelado em Sistemas de Informação.

Orientador: Dr. Lauro Víctor Ramos Cavadas

Niterói

2026

# **Sistema de Triagem de Doenças Foliares em Soja com Inteligência Artificial e Visão Computacional**

Rafael Ferreira Lucena - 1013176

Trabalho de conclusão de curso apresentado ao Centro Universitário La Salle do Rio de Janeiro (UNILASALLE-RJ) como requisito parcial à obtenção de grau de Bacharelado em Sistemas de Informação.

Banca Examinadora:

1. Orientador: Dr. Lauro Víctor Ramos Cavadas
2. Membro interno: Ms. Claudia Abreu Paes
3. Membro interno: Ms. João Paulo Baptista Voigtlaender

Niterói

2026

## Catálogo elaborado pelo Processamento Técnico da Biblioteca do Unilasalle-RJ



UNILASALLE - RJ  
CNPJ: 60.916.731/0026-61 - Inscrição Municipal: 112137-5  
Gastão Gonçalves, 79 - Santa Rosa  
Niterói/RJ - CEP: 24230030 - Telefone: 2199-6600

### Catálogo elaborado pelo Processamento Técnico da Biblioteca do Unilasalle-RJ

Lucena, Rafael Ferreira

Sistema de triagem de doenças foliares em soja com inteligência artificial e visão computacional / Rafael Ferreira Lucena - Niterói: Unilasalle-RJ 2026.  
54p. : il.

Orientador(a): Prof. Dr. Lauro Víctor Ramos Cavadas.  
Trabalho de conclusão de curso (Bacharelado em SISTEMAS DE INFORMAÇÃO) - Unilasalle-RJ - Centro Universitário La Salle-RJ.

1. Sistemas de informação - PROJETO. 2. Soja. 3. Doenças foliares. 4. Deep learning. 5. CBAM. API de inferência. I. Título.

CDD 004

Ficha catalográfica elaborada pela Bibliotecária Sabrina Ferreira G. Bera Carneiro CRB7/5687



Este documento eletrônico dispensa carimbo e assinatura.  
Sua autenticidade pode ser comprovada fornecendo o código de controle **4fb2898b-c9d2-40ed-8d53-85ec8c1745cb** na seguinte página da Rede La Salle  
<https://acesso.lasalle.edu.br/Corpore.Net/Source/Rpt-GeradorRelatoriosNet/RM.Rpt.Reports/Anonymous/RptFindReportByGuid.aspx> ou através do QRCode no corpo do relatório.

## RESUMO

A soja (*Glycine max* L.) ocupa posição estratégica na economia brasileira, com o agronegócio representando cerca de 29,4% do PIB nacional em 2025, sendo a cultura responsável por parcela expressiva das exportações do país. Apesar dessa relevância, estimativas técnicas apontam que doenças foliares comprometem entre 15% e 20% da produção anual, cenário agravado pela semelhança sintomatológica entre patologias distintas e pela limitada disponibilidade de assistência técnica especializada em campo. Diante disso, o presente trabalho propôs um sistema de triagem de doenças foliares em soja, com ênfase no rigor do modelo de aprendizado profundo e na eficiência de uma API de inferência. Uma interface frontend desenvolvida em HTML, CSS e JavaScript consome a API diretamente, exibindo a classe prevista, o nível de confiança e as barras de probabilidade para as oito classes. Os resultados experimentais evidenciaram que a arquitetura alcançou métricas altamente consistentes, registrando 94% de acurácia global e 94% de F1-score mediante a aplicação de Test-Time Augmentation (TTA), validando a viabilidade técnica da solução como serviço de apoio à decisão agrícola de baixo custo computacional.

**Palavras-chave:** Soja. Doenças Foliares. Deep Learning. CBAM. API de Inferência.

## ABSTRACT

Soybean (*Glycine max* L.) holds a strategic position in the Brazilian economy, with agribusiness accounting for approximately 29.4% of the national GDP in 2025, making the crop responsible for a significant share of the country's exports. Despite this relevance, technical estimates indicate that foliar diseases compromise between 15% and 20% of annual production, a scenario further worsened by the symptomatic similarity among distinct pathogens and the limited availability of specialized technical assistance in the field. In this context, the present work proposed a foliar disease triage system in soybean leaves, emphasizing the rigor of the deep learning model and the efficiency of an inference API. A frontend interface developed in HTML, CSS, and JavaScript consumes the API directly, displaying the predicted class, confidence level, and probability bars for all eight classes. Experimental results demonstrated that the proposed architecture achieved highly consistent metrics, recording 94% overall accuracy and a 94% F1-score upon the application of Test-Time Augmentation (TTA), validating the technical feasibility of the solution as a low computational cost service for agricultural decision support.

**Keywords:** Soybean. Foliar Diseases. Deep Learning. CBAM. Inference API.

# SUMÁRIO

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>1 - Introdução.....</b>                                                  | <b>12</b> |
| 1.1 - Motivação.....                                                        | 12        |
| 1.2 - Problema.....                                                         | 14        |
| 1.3 - Hipótese.....                                                         | 14        |
| 1.4 - Objetivos.....                                                        | 14        |
| 1.5 - Organização do Trabalho.....                                          | 15        |
| <b>2 - Referencial Teórico.....</b>                                         | <b>16</b> |
| 2.1 - Importância da Soja para o Brasil.....                                | 16        |
| 2.2 - Inteligência Artificial no Agronegócio.....                           | 17        |
| 2.3 - Doenças Foliares da Soja e Impactos no Agronegócio.....               | 18        |
| 2.4 - Visão Computacional e Detecção de Doenças em Plantas.....             | 20        |
| 2.5 - Ferramentas e Tecnologias Utilizadas.....                             | 21        |
| 2.6 - Trabalhos Relacionados.....                                           | 22        |
| <b>3 - Desenvolvimento.....</b>                                             | <b>25</b> |
| 3.1 - Dataset e Particionamento dos Dados.....                              | 25        |
| 3.2 - Pipeline de Pré-processamento e Data Augmentation com TensorFlow..... | 27        |
| 3.3 - Arquitetura da CNN V7 CBAM e Processo de Treinamento.....             | 29        |
| 3.3.1 - Estrutura da arquitetura.....                                       | 29        |
| 3.3.2 - O módulo CBAM.....                                                  | 30        |
| 3.3.3 - Cabeça classificadora.....                                          | 31        |
| 3.3.4 - Função de perda e configuração do treinamento.....                  | 31        |
| 3.3.5 - Callbacks e controle do treinamento.....                            | 32        |
| 3.4 - Encapsulamento do Modelo em API de Inferência com FastAPI.....        | 32        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>4 - Resultados.....</b>                                   | <b>37</b> |
| 4.1 - Curvas de Treinamento.....                             | 37        |
| 4.2 - Avaliação sem TTA.....                                 | 39        |
| 4.3 - Avaliação com TTA.....                                 | 41        |
| 4.4 - Análise dos Principais Erros de Classificação.....     | 43        |
| 4.4.1 - Confusão entre soybean_rust e frogeye.....           | 43        |
| 4.4.2 - Confusão entre cercospora_leaf_blight e healthy..... | 45        |
| 4.4.3 - Demais padrões de confusão.....                      | 47        |
| <b>5 - Conclusão.....</b>                                    | <b>48</b> |
| 5.1 - Considerações Finais.....                              | 48        |
| 5.2 - Limitações e Trabalhos Futuros.....                    | 49        |
| <b>Referências Bibliográficas.....</b>                       | <b>51</b> |



## LISTA DE FIGURAS

|                                                                                                 |           |
|-------------------------------------------------------------------------------------------------|-----------|
| <b>Figura 1:</b> Interface SojaVision — tela inicial com campo de upload.....                   | <b>34</b> |
| <b>Figura 2:</b> Interface SojaVision — resultado da inferência com patologia detectada.....    | <b>35</b> |
| <b>Figura 3:</b> Interface SojaVision — resultado da inferência com folha saudável.....         | <b>36</b> |
| <b>Figura 4:</b> Evolução da acurácia — CNN V7 CBAM (treino vs. validação).....                 | <b>37</b> |
| <b>Figura 5:</b> Evolução do erro (loss) — CNN V7 CBAM (treino vs. validação).....              | <b>38</b> |
| <b>Figura 6:</b> Matriz de Confusão sem TTA — conjunto de teste.....                            | <b>40</b> |
| <b>Figura 7:</b> Matriz de Confusão com TTA — conjunto de teste.....                            | <b>42</b> |
| <b>Figura 8:</b> Painel de erros — soybean_rust classificado como frog-eye (n=18).....          | <b>44</b> |
| <b>Figura 9:</b> Painel de erros — cercospora_leaf_blight classificado como healthy (n=11)..... | <b>46</b> |

## LISTA DE TABELAS

|                                                                                        |           |
|----------------------------------------------------------------------------------------|-----------|
| <b>Tabela 1:</b> Comparativo com os trabalhos relacionados.....                        | <b>23</b> |
| <b>Tabela 2:</b> Relatório de classificação sem TTA (conjunto de teste, n=1.448).....  | <b>39</b> |
| <b>Tabela 3:</b> Relatório de classificação com TTA (conjunto de teste , n=1.448)..... | <b>41</b> |

## LISTA DE ABREVIATURAS E SIGLAS

1. API – Interface de Programação de Aplicações (Application Programming Interface).
2. ASDID – Auburn Soybean Disease Image Dataset.
3. ASGI – Interface de Gateway de Servidor Assíncrono (Asynchronous Server Gateway Interface).
4. AUTOTUNE – Mecanismo de paralelismo automático da biblioteca TensorFlow.
5. CBAM – Módulo de Atenção Convolutiva (Convolutional Block Attention Module).
6. CNN – Rede Neural Convolutiva (Convolutional Neural Network).
7. CORS – Compartilhamento de Recursos de Origem Cruzada (Cross-Origin Resource Sharing).
8. CPU – Unidade Central de Processamento (Central Processing Unit).
9. CSV – Valores Separados por Vírgula (Comma-Separated Values).
10. GPU – Unidade de Processamento Gráfico (Graphics Processing Unit).
11. HTTP – Protocolo de Transferência de Hipertexto (HyperText Transfer Protocol).
12. IA – Inteligência Artificial.
13. JSON – Notação de Objeto JavaScript (JavaScript Object Notation).
14. MIME – Extensões de Correio da Internet Multipropósito (Multipurpose Internet Mail Extensions).
15. PIB – Produto Interno Bruto.
16. RGB – Vermelho, Verde e Azul (Red, Green, Blue).
17. TTA – Aumento de Dados em Tempo de Teste (Test-Time Augmentation).

# 1 - Introdução

O agronegócio tem papel central na economia brasileira e, em 2025, projeções indicaram que o setor pode representar cerca de 29,4% do PIB do Brasil, evidenciando sua relevância para a geração de renda e estabilidade econômica (CNA; CEPEA, 2025). Dentro desse cenário, a soja se destaca como uma cultura estratégica, com forte presença no comércio exterior brasileiro, tendo a China como principal destino, seguida por outros mercados como Espanha, Tailândia, Turquia, Irã e México, em diferentes períodos (REUTERS, 2025).

Apesar da alta importância, a cultura da soja enfrenta limitações relevantes, especialmente relacionadas a doenças, que podem ocorrer em diferentes fases do ciclo produtivo. Estimativas apontam perdas anuais de produção por doenças na faixa de 15% a 20% (SEIXAS et al., 2020), variando conforme a região e as condições climáticas, o que reforça a necessidade de estratégias que apoiem o monitoramento e a identificação precoce de sintomas. Nesse contexto, soluções baseadas em Inteligência Artificial (IA) e Visão Computacional podem apoiar a triagem inicial ao reconhecer padrões visuais em imagens de folhas, oferecendo uma classificação automática que auxilie no direcionamento do manejo.

Diante disso, este trabalho propôs e desenvolveu um sistema de triagem de doenças foliares em soja a partir de imagens, utilizando uma arquitetura de Rede Neural Convolucional customizada, projetada e treinada do zero, com integração nativa do Módulo de Atenção Convolucional (CBAM) para focalização nas regiões mais discriminativas das lesões foliares. A solução centralizou-se na construção de uma API de inferência otimizada capaz de receber a imagem foliar e retornar a classe prevista junto a um nível de confiança, validando a capacidade de generalização do modelo por meio de métricas consolidadas como acurácia e F1-score. Ressalta-se que a ferramenta desenvolvida teve caráter estrito de triagem e apoio ao manejo, não substituindo o diagnóstico técnico especializado em laboratório.

## 1.1 - Motivação

A cultura da soja possui grande relevância para o Brasil por seu impacto econômico e social, contribuindo para a geração de renda, empregos e fortalecimento da balança comercial. Projeções divulgadas pela Confederação da Agricultura e Pecuária do Brasil em

parceria com o Cepea/Esalq/USP indicam que, em 2025, o agronegócio pode representar aproximadamente 29,4% do PIB brasileiro, sinalizando a importância de iniciativas que contribuam para manter a produtividade e reduzir perdas ao longo da cadeia agrícola (CNA; CEPEA, 2025).

Nesse cenário, a soja consolida-se como uma das principais commodities do país, impulsionada por uma demanda internacional contínua e aquecida. Em 2025, a China permanece como o principal parceiro comercial, concentrando aproximadamente 80% das exportações totais de soja do Brasil, segundo dados citados pela ANEC em reportagens de mercado (REUTERS, 2025). Esse panorama reforça que a garantia de estabilidade produtiva e a preservação da sanidade dos grãos constituem pilares estratégicos para a competitividade do setor.

Apesar dessa importância fundamental, o rendimento das lavouras de soja é frequentemente ameaçado por desafios fitossanitários. Estudos técnicos indicam que as perdas anuais de produção decorrentes de doenças podem ser estimadas em torno de 15% a 20%, variando conforme a região e as condições climáticas de cada safra, o que evidencia a necessidade de monitoramento e detecção precoce (SEIXAS et al., 2020). Adicionalmente, o diagnóstico visual a olho nu no campo é frequentemente dificultado pela alta similaridade dos sintomas iniciais entre diferentes doenças, além de variações severas de iluminação ambiental e restrições de acesso imediato à assistência técnica especializada.

Diante disso, o amadurecimento da Inteligência Artificial aplicada ao agronegócio criou uma oportunidade ímpar para o emprego de algoritmos de Visão Computacional capazes de realizar uma triagem primária robusta e padronizada. A adoção de técnicas avançadas de aprendizado profundo viabilizou a construção de uma arquitetura de Rede Neural Convolucional customizada, projetada e treinada do zero sobre o dataset ASDID, com integração nativa do Módulo de Atenção Convolucional (CBAM) para focalização automática nas regiões sintomáticas das lesões foliares, dispensando o recurso a modelos pré-treinados externos e conferindo ao trabalho maior controle arquitetural e transparência metodológica (ABADI et al., 2016). A pesquisa justificou-se pelo interesse na interseção entre a biologia e a tecnologia, além do forte alinhamento com a área de Sistemas de Informação, ao envolver a construção de um pipeline de dados complexo com TensorFlow, a modelagem de aprendizado

de máquina do zero e a arquitetura de uma API de inferência em backend com FastAPI para suporte analítico à tomada de decisão agrícola.

## 1.2 - Problema

Este trabalho visou a resolver a seguinte questão científica e tecnológica:

**“Como apoiar a identificação de doenças foliares em soja a partir de imagens de folhas, de forma acessível e com maior precisão, considerando as dificuldades do diagnóstico visual em campo e a semelhança entre sintomas?”**

## 1.3 - Hipótese

A identificação de doenças foliares em soja pode ser auxiliada por um sistema de triagem baseado em Inteligência Artificial e Visão Computacional, no qual uma imagem da folha é submetida a uma API de inferência e o sistema retorna a doença provável acompanhada de um nível de confiança. Supõe-se que, ao utilizar uma arquitetura de Rede Neural Convolucional customizada, projetada e treinada do zero com integração nativa do Módulo de Atenção Convolucional (CBAM) sobre um conjunto de imagens representativo, o modelo seja capaz de classificar as principais doenças foliares com desempenho satisfatório, mensurado por métricas de acurácia e F1-score, e que o encapsulamento desse modelo em um serviço de backend desenvolvido com FastAPI constitua uma solução técnica viável para apoio à decisão no manejo fitossanitário.

## 1.4 - Objetivos

O objetivo deste trabalho foi desenvolver e avaliar um sistema de triagem de doenças foliares em folhas de soja, utilizando Inteligência Artificial e Visão Computacional, capaz de receber uma imagem da folha e retornar a classe prevista entre um conjunto delimitado de categorias, além de um nível de confiança. O sistema foi delimitado às oito classes do dataset ASDID, correspondentes às principais doenças foliares catalogadas, e à classe "saudável", mantendo o escopo técnico viável e adequado ao contexto do trabalho.

Para atingir esse objetivo, foi construída uma arquitetura de Rede Neural Convolucional (CNN) customizada, projetada e treinada do zero com integração nativa do Módulo de Atenção Convolucional (CBAM), sem recurso a modelos pré-treinados externos.

Todo o pipeline de dados foi implementado de forma nativa com TensorFlow, abrangendo carregamento, pré-processamento, normalização e data augmentation. O modelo foi avaliado por métricas de acurácia, precisão, revocação e F1-score, com análise da matriz de confusão para identificação dos principais erros de classificação entre classes semelhantes. Como produto final de software, foi entregue uma API de inferência de alta performance desenvolvida com FastAPI, capaz de receber requisições HTTP com a imagem foliar e retornar a classe prevista acompanhada do nível de confiança do modelo.

Como objetivo secundário, foi desenvolvida uma interface web de demonstração que permite ao usuário enviar uma imagem de folha de soja e visualizar o diagnóstico retornado pelo modelo, incluindo a classe prevista em português, o nível de confiança e as probabilidades para todas as oito classes do ASDID. Essa interface teve como propósito evidenciar a viabilidade de consumo da API por uma camada de apresentação acessível, aproximando o sistema do contexto de uso real pelo produtor rural ou técnico agrícola.

## **1.5 - Organização do Trabalho**

Este trabalho está organizado em cinco capítulos. O Capítulo 1 apresenta a introdução do trabalho, contextualizando a importância da soja para a economia brasileira, o problema das doenças foliares e a motivação para o desenvolvimento do sistema proposto, além de enunciar a questão-problema, a hipótese central, os objetivos geral e secundário e a organização dos capítulos. O Capítulo 2 apresenta o referencial teórico sobre a cultura da soja, suas patologias, fundamentos de IA e Visão Computacional, tecnologias empregadas e a revisão dos trabalhos relacionados. O Capítulo 3 descreve o desenvolvimento do sistema, detalhando o dataset utilizado e seu particionamento, o pipeline de pré-processamento com TensorFlow, a arquitetura da CNN V7 CBAM projetada e treinada do zero, e o encapsulamento do modelo em uma API de inferência com FastAPI. O Capítulo 4 apresenta os resultados obtidos, incluindo a análise das curvas de treinamento, a avaliação do modelo sem e com Test-Time Augmentation, e a análise qualitativa dos principais erros de classificação observados. O Capítulo 5 apresenta as considerações finais, discutindo o alcance dos objetivos propostos, as limitações identificadas e os direcionamentos para trabalhos futuros. Por fim, são apresentadas as referências bibliográficas utilizadas ao longo do trabalho.

## 2 - Referencial Teórico

Este capítulo está organizado em seis seções, abrangendo os principais conceitos e estudos relacionados ao tema do trabalho. Na seção 2.1, discute-se a importância da cultura da soja para o Brasil, destacando seu papel econômico e os desafios produtivos. Na seção 2.2, apresenta-se a aplicação da Inteligência Artificial (IA) no agronegócio, em especial no contexto da agricultura de precisão e sustentável. Em seguida, a seção 2.3 aborda as principais doenças foliares da soja e seus impactos na produtividade e economia do setor. A seção 2.4 trata da Visão Computacional, explicando seus fundamentos e como essa técnica pode auxiliar na identificação de doenças em plantas, com destaque para o uso de Redes Neurais Convolucionais (CNNs). A seção 2.5 descreve as ferramentas e tecnologias empregadas no desenvolvimento da arquitetura CNN com CBAM e da API de inferência em backend para triagem de doenças foliares em soja. Por fim, a seção 2.6 revisa os trabalhos relacionados e destaca os diferenciais desta pesquisa frente ao estado da arte.

### 2.1 - Importância da Soja para o Brasil

A soja é considerada um dos pilares do agronegócio brasileiro, sendo cultivada em larga escala e contribuindo de forma significativa para a economia nacional. O Brasil posiciona-se entre os maiores produtores e exportadores de soja do mundo, com uma área plantada estimada em 47,61 milhões de hectares na safra 2024/25 (CONAB, 2024). Esse volume expressivo de cultivo reflete a relevância da commodity, tanto para suprir a demanda interna quanto para atender ao mercado externo. No cenário das exportações, a China destaca-se como o principal destino da soja brasileira, absorvendo historicamente a maior parte dos embarques totais do país. Essa concentração evidencia a forte dependência do mercado chinês, tornando fundamental a manutenção da produtividade e da qualidade do grão para garantir a competitividade do Brasil no comércio global (REUTERS, 2025).

Vale ressaltar que o agronegócio como um todo possui grande peso na economia brasileira; projeções indicam que o setor agropecuário mantém participação expressiva no PIB nacional (CNA; CEPEA, 2025). Dentro desse contexto, a soja contribui significativamente para a geração de divisas, empregos e desenvolvimento regional, dada sua cadeia produtiva abrangente (produção, processamento, transporte e exportação). Manter a estabilidade da



produção e reduzir perdas ao longo dessa cadeia são estratégias cruciais para sustentar o crescimento do setor e atender à demanda mundial por soja de forma consistente (SEIXAS et al., 2020).

Apesar de sua importância econômica, a cultura da soja enfrenta desafios consideráveis em termos de sanidade vegetal. Diversas doenças foliares podem acometer as lavouras e comprometer seriamente a produtividade. Estimativas técnicas apontam que, em condições típicas de campo, as perdas anuais causadas por doenças variam significativamente, podendo chegar a casos extremos de perda total quando o manejo adequado não é realizado (HENNING et al., 2014). Entre essas enfermidades, destaca-se a ferrugem-asiática da soja, considerada a mais devastadora: há relatos de que sua incidência severa pode acarretar perdas de até 90% na colheita caso não haja controle eficaz (GODOY et al., 2023).

Outro aspecto desafiador é a dificuldade de identificação visual dessas doenças no campo. Devido ao grande número de patógenos que atacam a soja e à similaridade dos sintomas iniciais entre diferentes doenças, muitas vezes é complicado para o produtor reconhecer corretamente qual enfermidade está presente apenas pela observação a olho nu. Fatores ambientais, como variações de iluminação e estágio de desenvolvimento da planta, também podem mascarar ou imitar sintomas, confundindo o diagnóstico e atrasando a intervenção necessária, o que pode resultar em danos irreversíveis à lavoura (BARBEDO, 2013).

## **2.2 - Inteligência Artificial no Agronegócio**

A Inteligência Artificial (IA) é uma área da computação que busca desenvolver sistemas capazes de realizar tarefas que normalmente demandariam inteligência humana, como reconhecimento de padrões, tomada de decisão e previsão de eventos. Nos últimos anos, a IA tem encontrado ampla aplicação no agronegócio, impulsionando o que se conhece por agricultura de precisão e agricultura digital. Essa abordagem inovadora integra tecnologias avançadas incluindo sensores, análise de dados e algoritmos de aprendizado de máquina com o objetivo de melhorar a eficiência, a produtividade e a sustentabilidade das atividades agrícolas (GEOAGRI, [s.d.]).

Um exemplo prático é o monitoramento contínuo de lavouras por meio de sensores em solo, estações meteorológicas, imagens de satélite ou drones. Com esses dados,

algoritmos inteligentes conseguem detectar precocemente a ocorrência de pragas ou doenças, permitindo uma resposta rápida e direcionada. Esse tipo de aplicação de IA traz diversos benefícios: reduz a necessidade de aplicação indiscriminada de defensivos, diminui custos operacionais e mitiga o impacto ambiental do cultivo. De fato, estratégias baseadas em IA vêm ao encontro de objetivos de sustentabilidade, permitindo produzir mais com menos, ou seja, elevar a produtividade da soja ao mesmo tempo em que se utiliza de forma otimizada insumos como água, fertilizantes e pesticidas (CARVALHO, 2024).

Ademais, a IA no agronegócio não se limita ao controle de doenças e pragas. Ela abrange uma gama de aplicações, como previsão de safras, automação de máquinas agrícolas e classificação de qualidade de grãos por visão computacional. Técnicas de visão computacional já eram apontadas desde a década de 2000 como promissoras para inspeção de qualidade de alimentos (BROSNAN; SUN, 2004). Com o avanço do aprendizado profundo (deep learning), essas aplicações tornaram-se ainda mais precisas, dado que algoritmos modernos podem aprender diretamente com os dados brutos e identificar padrões complexos sem necessidade de programação de regras explícitas (DENG et al., 2014).

Conforme Deng et al. (2014), os métodos de aprendizado profundo distinguem-se por permitir que a máquina extraia características de alto nível dos dados através de múltiplas camadas de neurônios artificiais. Essa capacidade é particularmente útil na agricultura, onde imagens de plantas podem apresentar grande variabilidade. Redes neurais profundas, incluindo as convolucionais, conseguem reconhecer padrões sutis como pequenas manchas ou diferenças de coloração em folhas que indicam o início de uma doença. Assim, iniciativas que integrem IA ao manejo agrícola tendem a aprimorar a sustentabilidade econômica e ambiental da produção, alinhando-se às exigências contemporâneas de eficiência e responsabilidade ecológica (BARBEDO, 2019).

## **2.3 - Doenças Foliares da Soja e Impactos no Agronegócio**

A cultura da soja é suscetível a uma variedade de fitopatógenos que causam doenças em diferentes órgãos da planta, mas é nas folhas que ocorrem algumas das enfermidades mais danosas. Atualmente, já foram identificadas aproximadamente 40 doenças que acometem a soja no Brasil, com importâncias econômicas variáveis conforme a safra e a região (HENNING et al., 2014). Dentre as principais doenças foliares de origem fúngica que preocupam

produtores estão a ferrugem-asiática (*Phakopsora pachyrhizi*), o oídio (*Microsphaera diffusa*), a mancha-alvo (*Corynespora cassiicola*) e a mancha-parda (*Septoria glycines*), entre outras. Cada uma dessas enfermidades apresenta sintomatologia própria e requer estratégias de manejo específicas (GODOY et al., 2023).

Entre todas, a ferrugem-asiática da soja merece destaque especial por ser considerada a doença de maior potencial destrutivo na cultura. Detectada pela primeira vez no Brasil em 2001, ela rapidamente se espalhou pelas regiões produtoras. O fungo provoca desfolha precoce, o que impede a completa formação dos grãos e resulta em quedas acentuadas de produtividade (HENNING et al., 2014). Estudos da Embrapa indicam que, sem o controle adequado, a ferrugem pode acarretar perdas severas, chegando a inviabilizar a colheita em talhões inteiros. Por isso, esforços concentram-se no desenvolvimento de cultivares resistentes, uso rotativo de fungicidas e adoção do vazio sanitário (GODOY et al., 2023).

Outras doenças foliares também causam impactos relevantes. O oídio pode levar à redução na área fotossintética, impactando o enchimento de grãos, enquanto a mancha-alvo produz lesões necróticas que podem provocar desfolha. Embora individualmente essas doenças possam não causar perdas tão drásticas quanto a ferrugem em todas as safras, a presença simultânea de múltiplos patógenos exige um manejo integrado para manter a lavoura saudável (FIELDVIEW, 2020).

Do ponto de vista econômico, as consequências das doenças foliares vão além da redução direta na produtividade. A necessidade de aplicações adicionais de fungicidas eleva os custos de produção, diminuindo a margem de lucro. Além disso, grãos provenientes de plantas doentes podem apresentar qualidade inferior, resultando em desvalorização no mercado. Manejo inadequado e identificação tardia das enfermidades representam uma ameaça à estabilidade de toda a cadeia produtiva (HENNING et al., 2014). Nesse contexto, sistemas tecnológicos capazes de analisar imagens de folhas e apontar prováveis doenças surgem para suprir a demanda por agilidade e precisão no diagnóstico, servindo como suporte fundamental à tomada de decisão no manejo fitossanitário (BARBEDO, 2019).

## 2.4 - Visão Computacional e Detecção de Doenças em Plantas

A Visão Computacional (Computer Vision) é um ramo da ciência da computação dedicado ao desenvolvimento de métodos que permitam às máquinas extrair informações úteis de imagens. Em essência, trata-se de replicar em software a complexidade da percepção visual humana, realizando tarefas como reconhecimento de objetos e detecção de padrões (GRYFO, 2021). Um dos avanços mais importantes nessa área foi a aplicação de Redes Neurais Convolucionais (CNNs). Inspiradas no funcionamento do córtex visual biológico, conforme proposto inicialmente por Fukushima (1980) com o modelo Neocognitron, as CNNs modernas demonstraram desempenho superior em tarefas de classificação visual ao aprender representações hierárquicas dos dados (DENG et al., 2014).

No contexto de detecção de doenças em plantas, a visão computacional mediada por CNNs tem se mostrado uma abordagem promissora. Imagens digitais das folhas podem ser analisadas por algoritmos para identificar padrões sutis que caracterizam cada doença. Uma das vantagens dessa técnica é a consistência: um modelo treinado aplica o mesmo critério de forma repetitiva, superando a subjetividade da análise visual humana. Além disso, a capacidade de processar grandes volumes de imagens rapidamente viabiliza o monitoramento em larga escala (BEVERS et al., 2022).

Estudos recentes comprovam a efetividade das CNNs na tarefa de classificar doenças foliares em soja. Bevers et al. (2022), por exemplo, desenvolveram um modelo que atingiu alta acurácia na distinção entre várias doenças foliares utilizando a arquitetura DenseNet201, tendo também testado técnicas de transfer learning. Importa notar que o uso de imagens RGB comuns em métodos de visão computacional tem a vantagem de aproveitar câmeras acessíveis, como as de smartphones, facilitando a adoção da tecnologia pelos usuários finais (BEVERS et al., 2022).

A evolução dos sensores também ampliou as possibilidades da visão computacional no agro. O uso de sensores de profundidade e técnicas robustas de pré-processamento permitem que os sistemas atuais lidem melhor com condições adversas de campo, como iluminação variável e fundos complexos (HAN et al., 2013). Em suma, a integração de IA e visão computacional na detecção de doenças foliares tem o potencial de reduzir perdas por identificação tardia e orientar o uso de defensivos, atuando como um sistema de apoio à decisão que aumenta a eficiência e a sustentabilidade da produção de soja (BARBEDO, 2019).

## 2.5 - Ferramentas e Tecnologias Utilizadas

O desenvolvimento do sistema foi realizado integrando ferramentas de código aberto, priorizando um ambiente robusto para a escrita do código e eficiência na execução dos algoritmos de aprendizado profundo. As principais tecnologias utilizadas foram:

- **Python e Visual Studio Code:** a linguagem Python foi utilizada como base para todo o projeto, estruturada e desenvolvida dentro do editor de código Visual Studio Code (VS Code), que forneceu as ferramentas necessárias para escrita, depuração e organização do código.
- **TensorFlow e Keras:** bibliotecas de aprendizado profundo empregadas na construção, treinamento e validação da Rede Neural Convolucional customizada. Todo o pipeline de dados, incluindo carregamento, pré-processamento, normalização, redimensionamento e data augmentation, foi implementado de forma nativa por meio dos módulos `tf.data`, `tf.io` e `tf.image`, com paralelismo gerenciado via AUTOTUNE (ABADI et al., 2016).
- **CBAM (Convolutional Block Attention Module):** módulo de atenção convolucional implementado do zero e integrado nativamente à arquitetura CNN customizada, composto por um módulo de atenção de canal e um módulo de atenção espacial aplicados sequencialmente após cada bloco convolucional, direcionando o aprendizado da rede para as regiões mais discriminativas das lesões foliares (WOO et al., 2018).
- **FastAPI:** framework web de alta performance utilizado para encapsular o modelo treinado em uma API de inferência de backend, capaz de receber requisições HTTP com a imagem foliar e retornar a classe prevista acompanhada do nível de confiança do modelo.
- **HTML, CSS e JavaScript:** tecnologias web utilizadas no desenvolvimento do frontend da aplicação, responsável por consumir a API de inferência e apresentar os resultados ao usuário de forma visual e interativa, com exibição

da classe prevista em português, nível de confiança e barras de probabilidade animadas para as oito classes do ASDID.

Essas ferramentas foram integradas para construir uma arquitetura modular e eficiente, centrando o esforço técnico no rigor do modelo de aprendizado profundo, na performance do serviço de inferência e na acessibilidade da interface de demonstração.

## 2.6 - Trabalhos Relacionados

A crescente aplicação de Visão Computacional e aprendizado profundo para o diagnóstico automatizado de doenças foliares em plantas tem gerado uma série de trabalhos relevantes na literatura científica recente. A análise desses estudos permite contextualizar as escolhas metodológicas do presente trabalho e evidenciar seus diferenciais em relação ao estado da arte.

Yu et al. (2022) propuseram um método de reconhecimento de doenças foliares em soja baseado em uma rede de atenção residual denominada RANet, desenvolvida a partir da adaptação do modelo ResNet18 com a incorporação de camadas de atenção residual. O estudo utilizou três classes de doenças foliares, sendo elas mancha-parda, mancha olho-de-rã e mancha de filosticta, e aplicou o algoritmo OTSU para remoção de fundo das imagens, seguido de técnicas de aumento de dados para expansão do dataset. Os resultados reportados indicaram acurácia média de reconhecimento de 98,49% e F1-score de 98,52%, com tempo de inferência de 0,0514 segundos por imagem. A arquitetura proposta demonstrou que a integração de mecanismo de atenção em redes residuais pode melhorar significativamente a capacidade discriminativa do modelo para padrões foliares sutis (YU et al., 2022).

Bevers et al. (2022) desenvolveram um sistema de identificação de doenças foliares em soja utilizando imagens reais de campo capturadas em condições não controladas, empregando a arquitetura profunda DenseNet201 treinada do zero. Os autores testaram também técnicas de *transfer learning*, que não produziram melhora significativa nos resultados em comparação ao treinamento integral dos pesos da rede. O estudo se destacou por introduzir e utilizar o *Auburn Soybean Disease Image Dataset* (ASDID), o mesmo dataset adotado no presente trabalho, e por demonstrar que arquiteturas profundas conseguem atingir alta acurácia de 96,8% mesmo com imagens capturadas em condições adversas de

iluminação e fundo complexo, ressaltando a viabilidade do uso de câmeras RGB acessíveis, como as de smartphones, para o diagnóstico fitopatológico em campo (BEVERS et al., 2022).

**Tabela 1:** Comparativo com os trabalhos relacionados.

| Critério              | Yu et al. (2022)            | Bevers et al. (2022)  | Presente Trabalho (2026) |
|-----------------------|-----------------------------|-----------------------|--------------------------|
| Arquitetura:          | RANet (ResNet18 modificada) | DenseNet201           | CNN customizada do zero  |
| Mecanismo de atenção: | Atenção residual (RAL)      | Não utiliza           | CBAM (canal + espacial)  |
| Dataset:              | Próprio 3 classes           | ASDID 8 classes       | ASDID 8 classes          |
| Acurácia:             | 98,49%                      | 96,8%                 | 92% / 94% com TTA        |
| Transfer Learning     | Sim                         | Testado, não melhorou | Não                      |
| Volume de imagens     | Não especificado            | ~9.500 imagens        | 9.648 imagens            |
| F1-score              | 98,52%                      | Não reportado         | 0,94 (macro)             |
| Avaliação             | Padrão                      | Padrão                | TTA com 5 variantes      |
| Classes avaliadas     | 3                           | 8                     | 8                        |

**Fonte:** Autoria própria (2026).

A Tabela 1 sintetiza as principais características dos estudos revisados, contrastando-as com a metodologia adotada nesta pesquisa. Conforme ilustrado, em comparação com os trabalhos relacionados, o presente estudo apresenta dois diferenciais principais. Primeiro, enquanto Yu et al. (2022) adaptaram uma rede consagrada e Bevers et al. (2022) treinaram do zero uma arquitetura pré-existente de alta complexidade (DenseNet201), este trabalho projetou e treinou uma estrutura de CNN própria e customizada do zero, conferindo controle total sobre cada decisão arquitetural, redução no custo computacional e maior transparência metodológica. Segundo, a integração nativa do CBAM (*Convolutional Block Attention Module*) após cada bloco convolucional representa uma abordagem de atenção mais granular do que a utilizada por Yu et al. (2022), que incorporou atenção apenas em camadas residuais

específicas, e inédita no cenário avaliado por Bevers et al. (2022). Além disso, é importante contextualizar os resultados reportados nos trabalhos relacionados em relação aos obtidos no presente estudo. A acurácia de 98,49% alcançada por Yu et al. (2022) foi obtida sobre um problema de apenas três classes em dataset próprio com condições controladas de captura, o que representa uma tarefa significativamente mais simples do que a classificação de oito classes em imagens de campo real. Já a comparação com Bevers et al. (2022) é a mais direta e relevante, pois ambos os estudos utilizaram o mesmo dataset ASDID com as mesmas oito classes. A diferença de 2,8 pontos percentuais de acurácia em favor de Bevers et al. (2022) é justificável pelo uso da DenseNet201, arquitetura de alta complexidade com significativamente mais parâmetros do que a CNN leve desenvolvida no presente trabalho. Essa diferença marginal de desempenho, obtida com uma arquitetura customizada de menor custo computacional e com a integração do CBAM, reforça a eficiência da solução proposta.

Dessa forma, a análise do estado da arte evidencia a viabilidade e a relevância da abordagem adotada nesta pesquisa, demonstrando que há espaço para contribuições que equilibrem alta capacidade preditiva com eficiência computacional. O capítulo a seguir apresenta o desenvolvimento do projeto, detalhando o ambiente tecnológico utilizado, os processos de preparação do dataset ASDID, a concepção arquitetural da CNN customizada e a implementação do módulo de atenção CBAM.



## 3 - Desenvolvimento

Neste capítulo é descrito o processo de desenvolvimento do sistema proposto, organizado em quatro seções principais. A seção 3.1 apresenta o dataset utilizado e as etapas de preparação e particionamento dos dados. A seção 3.2 descreve a construção do pipeline de pré-processamento e data augmentation, implementado de forma nativa com TensorFlow. A seção 3.3 detalha a arquitetura da Rede Neural Convolucional customizada (CNN V7 CBAM), explicando as decisões técnicas que motivaram cada componente, incluindo a integração do Módulo de Atenção Convolucional (CBAM), e o processo de treinamento com Focal Loss e os mecanismos de controle de overfitting. Por fim, a seção 3.4 aborda o encapsulamento do modelo treinado em uma API de inferência de backend desenvolvida com FastAPI, constituindo o produto final de software do trabalho.

O desenvolvimento foi conduzido integralmente em Python, com o ambiente de desenvolvimento estruturado no Visual Studio Code. A escolha por construir a arquitetura do zero, sem recurso a modelos pré-treinados externos, teve como motivação central o controle total sobre cada decisão arquitetural, tornando o processo de aprendizagem e documentação mais transparente e rigoroso. Todas as etapas, desde o carregamento das imagens até a exposição do modelo via API, foram implementadas de forma modular e reproduzível, priorizando eficiência computacional e clareza metodológica.

### 3.1 - Dataset e Particionamento dos Dados

O dataset utilizado neste trabalho foi o Auburn Soybean Disease Image Dataset (ASDID), base pública amplamente validada na literatura de visão computacional aplicada à fitopatologia (BEVERS et al., 2022). O ASDID é composto por 9.648 imagens de folhas de soja coletadas em condições reais de campo e de laboratório, distribuídas em oito categorias: bacterial blight, cercospora leaf blight, downey mildew, frogeye leaf spot, healthy, potassium deficiency, soybean rust e target spot. A escolha desse dataset justificou-se tanto pela sua representatividade, com imagens capturadas em diferentes condições de iluminação, ângulos e backgrounds, quanto por sua adoção recorrente como benchmark em estudos recentes de classificação de doenças foliares em soja (ZHANG et al., 2025; LI et al., 2026).

A análise exploratória do dataset revelou um desequilíbrio relevante na distribuição de classes: a categoria bacterial blight contava com apenas 484 imagens originais, enquanto healthy possuía 1.632, uma proporção superior a 3:1. Esse desequilíbrio, caso ignorado, tenderia a enviesar o modelo para as classes mais representadas, penalizando o desempenho nas menos frequentes. Para mitigar esse problema, foi aplicado um processo de data augmentation offline, gerado previamente ao treinamento e consolidado em arquivos CSV de split. Nesse processo, cada imagem das classes minoritárias foi submetida a transformações geométricas e fotométricas, incluindo rotação, ajuste de contraste e redimensionamento com letterbox, de forma a expandir o volume total do conjunto de dados para aproximadamente 18 mil amostras, com distribuição inter-classes mais equilibrada. Esse conjunto aumentado foi então persistido em disco e referenciado por um arquivo CSV estruturado com três colunas: path (caminho da imagem), label (rótulo da classe) e split (partição de destino), viabilizando a reprodutibilidade integral dos experimentos.

O particionamento do dataset foi realizado seguindo a proporção 7:1:2, com semente aleatória fixada em seed=42 para garantir reprodutibilidade. Essa decisão foi motivada por sua aderência à estratégia adotada pelo artigo de referência utilizado como fundamentação arquitetural (ZHANG et al., 2025), que também empregou o ASDID com divisão 7:1:2. Do total de amostras aumentadas, aproximadamente 70% foram destinadas ao treinamento, 10% à validação durante o processo de treinamento e 20% ao conjunto de teste para avaliação final, resultando em 1.448 amostras no conjunto de teste, as quais foram utilizadas de forma isolada para a extração definitiva das métricas registradas nos relatórios de classificação do Experimento 16. Durante o carregamento, o arquivo CSV foi lido com a biblioteca Pandas e filtrado pela coluna split para separar os subconjuntos de treino e validação, com os rótulos convertidos para índices numéricos por mapeamento direto e posteriormente codificados em formato one-hot para compatibilidade com a função de perda adotada.

## 3.2 - Pipeline de Pré-processamento e Data Augmentation com TensorFlow

Todo o processo de carregamento, pré-processamento e augmentation em tempo de treinamento foi implementado de forma nativa utilizando a API `tf.data` do TensorFlow, sem dependência de bibliotecas externas de visão computacional. Essa decisão teve motivação técnica direta: a API `tf.data` permite construir pipelines de dados paralelizados e otimizados em grafo computacional, integrando-se ao motor de execução do TensorFlow e eliminando a necessidade de pré-processamento em CPU desacoplado do treinamento (ABADI et al., 2016). O paralelismo foi configurado com `tf.data.AUTOTUNE` em todas as operações de mapeamento, delegando ao runtime do TensorFlow a decisão sobre o número ótimo de threads paralelos de acordo com os recursos disponíveis do hardware.

O carregamento de cada imagem foi implementado na função `load_and_preprocess`, que recebe o caminho do arquivo e o rótulo codificado como entradas. Internamente, a imagem é lida em bytes com `tf.io.read_file`, decodificada como JPEG em três canais RGB com `tf.io.decode_jpeg` e, em seguida, redimensionada para a resolução padrão de 224×224 pixels com `tf.image.resize` caso não esteja nessa dimensão, verificação feita condicionalmente via `tf.cond` para evitar redimensionamentos desnecessários e preservar eficiência. A normalização foi aplicada diretamente no tensor, dividindo todos os valores de pixel pelo escalar 255.0, mapeando o intervalo [0, 255] para [0.0, 1.0]. Essa etapa é fundamental para a estabilidade numérica do gradiente durante o treinamento e é considerada prática padrão em CNNs aplicadas à classificação de imagens (DENG et al., 2014).

Para o conjunto de treinamento, foi aplicado data augmentation online por meio de uma camada `keras.Sequential` composta por quatro transformações aleatórias aplicadas em tempo de execução a cada época:

- **Espelhamento horizontal aleatório:** inversão do eixo horizontal da imagem (`RandomFlip "horizontal"`), simulando diferentes orientações de captura da folha.
- **Rotação aleatória:** inclinação de até 20 graus (`RandomRotation`), reproduzindo variações angulares comuns na fotografia em campo.

- **Translação aleatória:** movimentação de até 10% nos eixos horizontal e vertical (RandomTranslation), simulando enquadramentos distintos da câmera em relação à folha.
- **Zoom aleatório:** aproximação ou distanciamento de até 10% (RandomZoom), representando variações de distância entre o dispositivo de captura e a superfície foliar.

Essas transformações foram escolhidas por simularem variações reais de captura em campo sem introduzir distorções irrealistas que pudessem prejudicar a generalização do modelo. Diferentemente do augmentation offline realizado na etapa de preparação do dataset, esse augmentation online garante que o modelo veja versões ligeiramente diferentes de cada imagem a cada época, aumentando efetivamente a diversidade dos dados de treinamento sem custo adicional de armazenamento. Ao conjunto de validação não foi aplicado nenhum augmentation, apenas o carregamento e a normalização padrão garantindo que as métricas de validação reflitam o desempenho real do modelo sobre dados não alterados.

A construção do dataset seguiu a função `make_dataset`, que organiza o pipeline completo: criação do dataset TensorFlow a partir dos pares caminho-rótulo com `tf.data.Dataset.from_tensor_slices`; embaralhamento aleatório completo do conjunto de treinamento a cada época com `buffer_size` igual ao total de amostras (`reshuffle_each_iteration=True`), evitando padrões de ordem que pudessem induzir viés no aprendizado; mapeamento paralelo das funções de carregamento e augmentation; agrupamento em batches de 32 imagens; e, por fim, pré-carregamento assíncrono com `prefetch(AUTOTUNE)`, que mantém a GPU ocupada enquanto o próximo batch é preparado em paralelo pela CPU. O cache em disco foi deliberadamente desativado para evitar esgotamento de memória RAM em ambientes com recursos limitados, decisão documentada diretamente no código-fonte. A label smoothing foi aplicada exclusivamente ao conjunto de treinamento com fator de 0.05, suavizando as distribuições de rótulo para reduzir a confiança excessiva do modelo nas predições, técnica conhecida por melhorar a calibração e a generalização em tarefas de classificação multiclasse (ABADI et al., 2016).

### 3.3 - Arquitetura da CNN V7 CBAM e Processo de Treinamento

A arquitetura apresentada nesta seção, denominada internamente CNN V7, foi o resultado de um processo iterativo de desenvolvimento que envolveu a concepção e avaliação de seis versões distintas de rede e dezesseis experimentos de treinamento. Cada iteração incorporou ajustes progressivos como mudanças na profundidade dos blocos, nos mecanismos de regularização e, por fim, na integração do módulo CBAM com base na análise das métricas de validação de cada experimento anterior. A versão aqui descrita corresponde à configuração que apresentou os melhores resultados consolidados, sendo denominada Experimento 16 (Exp16) no contexto do desenvolvimento deste trabalho.

#### 3.3.1 -Estrutura da arquitetura

A CNN V7 foi projetada e treinada do zero, sem recurso a pesos pré-treinados externos. A arquitetura é composta por três blocos convolucionais progressivos seguidos de uma cabeça classificadora, totalizando uma estrutura leve e adequada ao volume de dados disponível no ASDID. Cada bloco convolucional segue o mesmo padrão interno: duas camadas Conv2D com kernel 3×3, padding "same" e use\_bias=False o viés é suprimido pois o BatchNormalization subsequente já cumpre esse papel seguidas de BatchNormalization e ativação ReLU em cada camada, uma camada MaxPooling2D de 2×2 para redução espacial pela metade, um módulo CBAM aplicado sobre o mapa de características resultante e, por fim, uma camada SpatialDropout2D para regularização espacial. O número de filtros evolui progressivamente entre os blocos 32, 64 e 128 respectivamente capturando características de complexidade crescente: o Bloco 1 foca em padrões básicos como bordas e variações de cor; o Bloco 2 extrai texturas intermediárias da superfície foliar; e o Bloco 3 é responsável por padrões complexos como manchas, lesões e halos característicos das diferentes patologias.

A regularização foi calibrada de forma progressiva ao longo dos blocos, com taxas de SpatialDropout2D de 0.1, 0.15 e 0.2 respectivamente. O SpatialDropout2D foi preferido ao Dropout convencional por desligar mapas de características inteiros em vez de neurônios individuais, forçando a rede a desenvolver representações redundantes e mais robustas — comportamento especialmente importante em tarefas de classificação de lesões foliares onde diferentes filtros podem aprender características visualmente similares entre classes (ZHANG et al., 2025).

### 3.3.2 -O módulo CBAM

O Convolutional Block Attention Module (CBAM) foi implementado do zero como uma camada Keras customizada e integrado após cada bloco convolucional da arquitetura, operando sobre o mapa de características já reduzido pelo MaxPooling. O CBAM é composto por dois submódulos aplicados sequencialmente: o módulo de atenção de canal (Channel Attention) e o módulo de atenção espacial (Spatial Attention) (WOO et al., 2018).

O módulo de atenção de canal opera sobre a dimensão das features e aprende quais canais ou seja, quais tipos de características extraídas são mais relevantes para a classificação. Para isso, aplica tanto Global Average Pooling quanto Global Max Pooling sobre o mapa de entrada, processando ambas as saídas por uma rede densa compartilhada com fator de redução 4, somando os resultados e aplicando uma função sigmoide para gerar pesos de atenção por canal no intervalo  $[0,1]$ . Esses pesos são multiplicados elemento a elemento pelo mapa de características original, amplificando os canais mais discriminativos e suprimindo os menos relevantes.

O módulo de atenção espacial, aplicado em seguida, opera sobre a dimensão espacial e aprende onde na imagem prestar atenção. Calcula o average pooling e o max pooling ao longo da dimensão dos canais, concatena os dois mapas resultantes em um tensor de dois canais e aplica uma convolução 2D com kernel  $7 \times 7$  e um único filtro de saída, seguida de sigmoide, gerando um mapa de atenção espacial de dimensões  $H \times W \times 1$ . Esse mapa é então multiplicado pelo tensor de entrada, destacando as regiões espaciais mais informativas no contexto deste trabalho, as regiões onde as lesões foliares estão presentes.

A motivação para a integração do CBAM foi diretamente observacional: em experimentos anteriores ao Exp16, o modelo apresentava dificuldade em distinguir frog-eye leaf spot de folhas healthy, pois os sintomas iniciais do frog-eye pequenas manchas circulares com halo amarelado ocupam apenas uma fração pequena da área foliar e são facilmente mascarados pelo contexto verde saudável ao redor. O mecanismo de atenção espacial do CBAM endereçou esse problema ao direcionar o gradiente para as regiões lesionadas, melhorando o recall da classe frog-eye. Estudos recentes confirmam que a integração do CBAM em arquiteturas CNN para classificação de doenças foliares produz ganhos consistentes de acurácia e F1-score com custo paramétrico marginal (ZHANG et al., 2025).

### 3.3.3 -Cabeça classificadora

Após o terceiro bloco convolucional com CBAM, a rede diverge da abordagem convencional de Flatten e adota uma fusão dupla de pooling global. O mapa de características passa simultaneamente por GlobalAveragePooling2D que captura o contexto geral da folha, como coloração difusa e textura média e por GlobalMaxPooling2D que detecta o pico de ativação mais intenso, essencial para identificar lesões pequenas e localizadas. Os dois vetores resultantes são concatenados, formando uma representação que combina informação contextual e informação pontual. Essa representação passa por uma camada Dense de 256 unidades com ativação ReLU e regularização L2 de 0.001, seguida de Dropout de 0.3, e finalmente pela camada de saída Dense com 8 unidades e ativação Softmax, produzindo a distribuição de probabilidade sobre as oito classes do ASDID.

### 3.3.4 -Função de perda e configuração do treinamento

O modelo foi compilado com o otimizador Adam com taxa de aprendizado inicial de  $1e-4$ , escolha justificada pela sua adaptatividade por parâmetro e convergência estável em tarefas de classificação de imagens (KINGMA; BA, 2015). Como função de perda, foi adotada uma Focal Loss por classe com  $\gamma=2.0$ , variante da função proposta por Lin et al. (2017) que penaliza de forma adaptativa os exemplos mal classificados, atribuindo maior peso ao gradiente dos casos de erro e reduzindo a contribuição das predições já corretas. Isso é particularmente relevante no contexto do ASDID, onde algumas classes são intrinsecamente mais difíceis de distinguir do que outras.

A parametrização da Focal Loss foi refinada com vetores de alpha por classe, ajustados com base na análise dos erros de classificação dos experimentos anteriores. A classe *frogeye* recebeu  $\alpha=0.45$  o maior valor do vetor devido à sua tendência de ser confundida com a classe *healthy* em experimentos anteriores, onde suas lesões iniciais sutis resultavam em baixo recall. A classe *bacterial\_blight* recebeu  $\alpha=0.20$ , valor elevado em relação ao experimento anterior (0.15) para recuperar o recall dessa classe, que apresentava confusão com outras patologias de sintomatologia similar. As demais seis classes mantiveram  $\alpha=0.25$ .

### 3.3.5 -Callbacks e controle do treinamento

O processo de treinamento foi controlado por três callbacks: ModelCheckpoint com monitoramento da val\_loss e salvamento apenas do melhor modelo (save\_best\_only=True), garantindo que o arquivo final corresponda à época de menor perda de validação; EarlyStopping com paciência de 12 épocas e restauração automática dos melhores pesos (restore\_best\_weights=True), interrompendo o treinamento quando a val\_loss estagnasse por 12 épocas consecutivas sem melhora; e ReduceLROnPlateau com fator de redução 0.2, paciência de 4 épocas e learning rate mínimo de 1e-7, que reduzia a taxa de aprendizado em 80% sempre que a val\_loss estagnasse por 4 épocas, produzindo o efeito de ajuste fino progressivo sem necessidade de intervenção manual. A paciência elevada do EarlyStopping 12 épocas foi uma decisão deliberada, pois redes treinadas do zero tendem a apresentar períodos de aparente estagnação antes de retomar a convergência, e uma paciência menor poderia interromper o treinamento prematuramente. O treinamento foi executado com até 100 épocas máximas, sendo encerrado antecipadamente na época 48 pelo EarlyStopping, com tempo total de aproximadamente 1.160 minutos.

## 3.4 - Encapsulamento do Modelo em API de Inferência com FastAPI

O produto final de software do presente trabalho consistiu no encapsulamento do modelo CNN V7 CBAM treinado em um serviço de backend de alta performance desenvolvido com o framework FastAPI. A escolha do FastAPI justificou-se por três características técnicas centrais: suporte nativo a operações assíncronas via Python ASGI, geração automática de documentação interativa via OpenAPI, e desempenho superior em benchmarks de throughput em comparação a frameworks síncronos como Flask.

A aplicação foi estruturada em um único módulo main.py, organizado em quatro blocos funcionais. O primeiro bloco define as configurações globais: tamanho de entrada da imagem (224×224 pixels), caminho do modelo e os vetores de nomes e rótulos das oito classes em inglês e português. O segundo bloco implementa as três camadas customizadas do CBAM, sendo elas ChannelAttention, SpatialAttention e CBAM, como subclasses de tf.keras.layers.Layer, registradas em um dicionário CUSTOM\_OBJECTS. Esse registro é

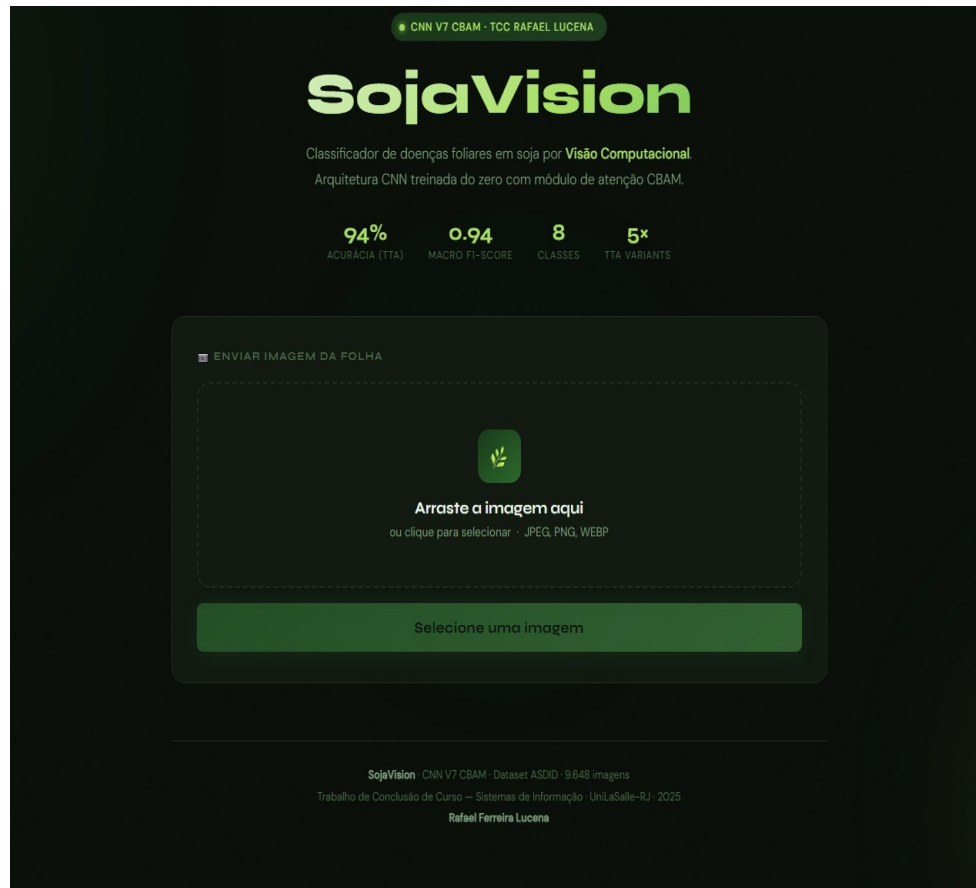


indispensável para que o TensorFlow consiga deserializar corretamente o modelo salvo, uma vez que as camadas de atenção não fazem parte do repertório nativo do Keras e precisam ser declaradas explicitamente no momento do carregamento via `custom_objects`. O modelo é carregado uma única vez na inicialização da aplicação com `tf.keras.models.load_model(MODEL_PATH, compile=False, custom_objects=CUSTOM_OBJECTS)`, permanecendo em memória durante toda a vida do servidor, estratégia que elimina a latência de carregamento por requisição.

O terceiro bloco implementa as funções auxiliares de inferência. A função `preprocess_image` recebe um objeto PIL Image, converte para RGB, redimensiona para 224×224 com interpolação Lanczos e normaliza os valores de pixel para o intervalo [0.0, 1.0], pipeline idêntico ao utilizado durante o treinamento. A função `apply_tta` gera as cinco variantes geométricas de Test-Time Augmentation: imagem original, flip horizontal, flip vertical, rotação de 90° e rotação de 270°. A função `predict_with_tta` empilha as cinco variantes em um batch, executa `model.predict` uma única vez sobre o batch completo, calcula a média das distribuições de probabilidade resultantes e retorna a classe prevista, o rótulo em português, o nível de confiança em porcentagem e o dicionário completo de probabilidades das oito classes.

O quarto bloco define as rotas HTTP da aplicação. A rota GET / serve o arquivo `index.html` diretamente, permitindo que o frontend seja acessado pelo mesmo servidor sem necessidade de infraestrutura separada. A rota GET /health retorna o status do servidor e confirma se o modelo foi carregado com sucesso, útil para verificação antes de demonstrações. A rota principal POST /predict recebe a imagem como multipart/form-data, valida o tipo MIME (JPEG, PNG ou WEBP), processa a imagem, executa a inferência com TTA e retorna o resultado em JSON. O middleware CORS foi configurado com `allow_origins=["*"]` para permitir requisições de qualquer origem, viabilizando o consumo da API por frontends externos. O servidor é inicializado com Uvicorn via `uvicorn main:app --reload`, ficando disponível em `http://localhost:8000`.

A interface frontend desenvolvida foi validada em ambiente local, demonstrando o funcionamento completo do sistema de triagem proposto. As figuras a seguir ilustram as principais telas da aplicação, evidenciando a integração entre o modelo de inferência e a camada de apresentação.

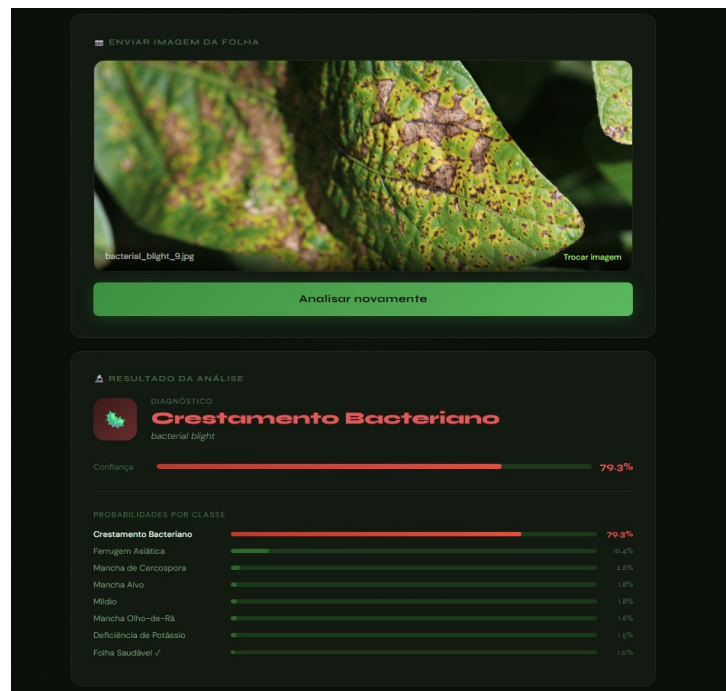


**Figura 1:** Interface SojaVision — tela inicial com campo de upload.

**Fonte:** Autoria própria (2026).

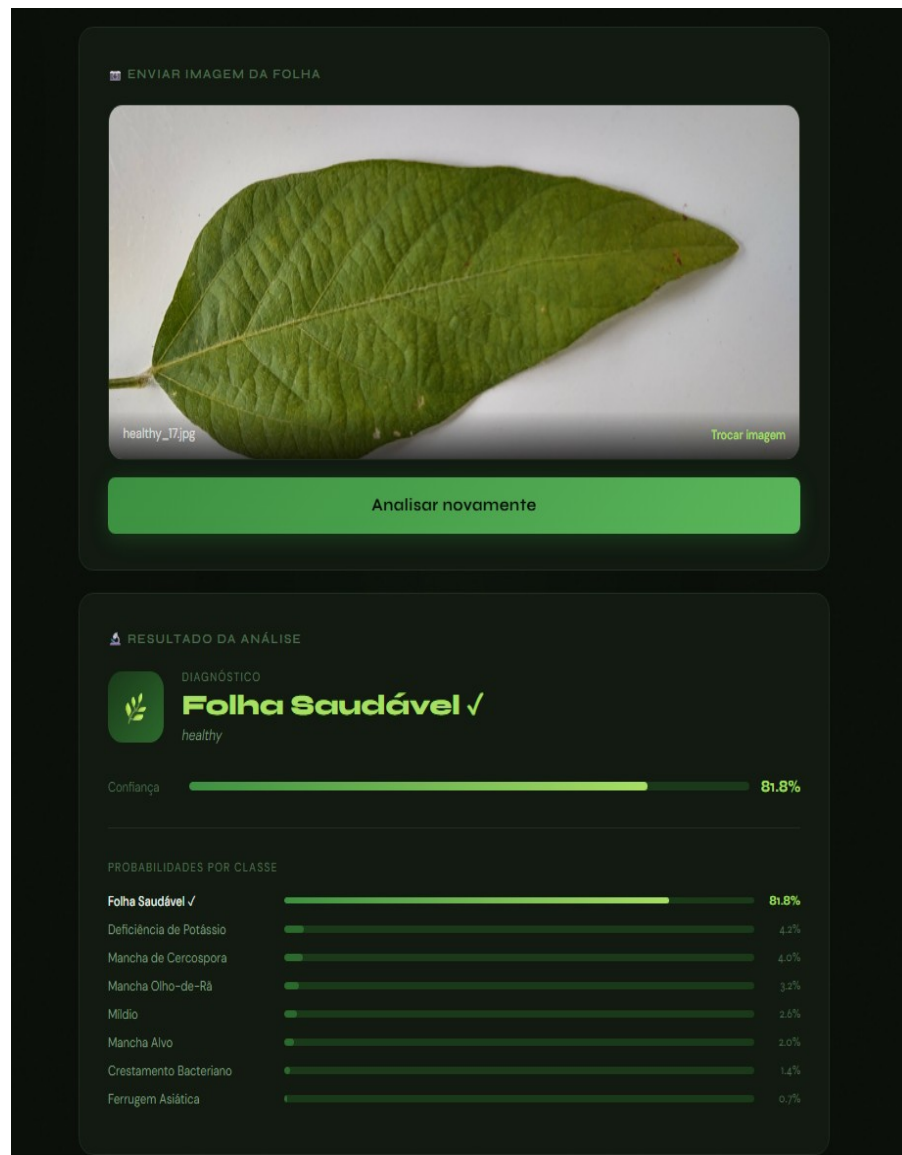
A tela inicial da aplicação, apresentada na Figura 1, exibe as métricas consolidadas do modelo, acurácia de 94%, macro F1-score de 0.94, oito classes e cinco variantes TTA, permitindo que o usuário compreenda o desempenho do sistema antes de realizar qualquer análise. A área central disponibiliza o campo de upload, com suporte a arrastar e soltar a imagem, tornando a interação intuitiva e acessível.

A Figura 2 ilustra um exemplo de resultado retornado pelo sistema após a inferência com TTA. A interface exibe de forma clara a classe prevista em português, o nível de confiança em porcentagem e as barras de probabilidade animadas para as oito classes do ASDID ordenadas da maior para a menor probabilidade, permitindo que o usuário compreenda não apenas o diagnóstico principal mas também o grau de certeza do modelo em relação às demais categorias.



**Figura 2:** Interface SojaVision — resultado da inferência com patologia detectada.  
**Fonte:** Autoria própria (2026).

A Figura 3 demonstra o comportamento do sistema para uma folha classificada como saudável, onde a interface adota a codificação visual em verde para indicar ausência de patologia detectada, em contraste com a codificação em vermelho utilizada para diagnósticos de doença, reforçando a clareza informacional da solução para o usuário final.



**Figura 3:** Interface SojaVision — resultado da inferência com folha saudável.

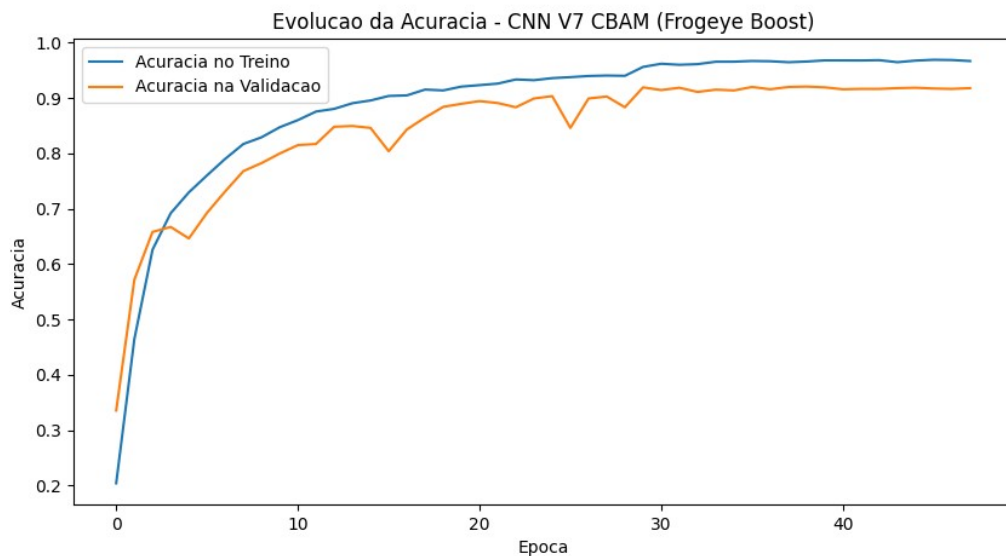
**Fonte:** Autoria própria (2026).

## 4 - Resultados

Neste capítulo são apresentados e discutidos os resultados obtidos com o treinamento e a avaliação da arquitetura CNN V7 CBAM no dataset ASDID. O capítulo está organizado em quatro seções: a seção 4.1 analisa as curvas de treinamento; a seção 4.2 apresenta os resultados da avaliação sem Test-Time Augmentation; a seção 4.3 apresenta os resultados com TTA e compara os ganhos obtidos; e a seção 4.4 realiza uma análise qualitativa dos principais erros de classificação observados.

### 4.1 - Curvas de Treinamento

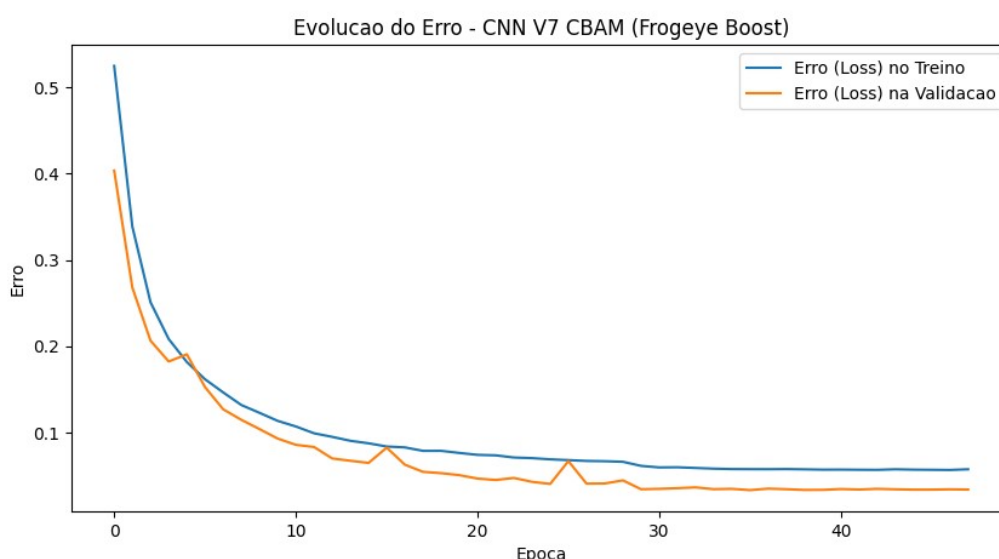
O treinamento da CNN V7 CBAM foi encerrado na época 48 pelo mecanismo de EarlyStopping, com tempo total de execução de aproximadamente 1.160 minutos. O limite máximo configurado era de 100 épocas, indicando que o modelo atingiu estagnação na val\_loss antes de esgotar o orçamento de treinamento comportamento esperado e desejado em redes treinadas do zero com regularização adequada.



**Figura 4:** Evolução da acurácia — CNN V7 CBAM (treino vs. validação).  
**Fonte:** Autoria própria (2026).

A análise da curva de acurácia, ilustrada na Figura 4, evidencia um padrão de aprendizado saudável e consistente ao longo de todo o treinamento. A acurácia de treino partiu de aproximadamente 22% na época 1 e convergiu progressivamente para cerca de 96%

na época final, enquanto a acurácia de validação acompanhou a curva de treino de forma próxima partindo de 33% e estabilizando em torno de 92% nas últimas épocas. A proximidade entre as duas curvas ao longo de todo o processo indica ausência de overfitting severo, demonstrando que os mecanismos de regularização adotados SpatialDropout2D progressivo, Dropout na cabeça classificadora e regularização L2, foram eficazes em conter a memorização excessiva do conjunto de treinamento. A diferença residual de aproximadamente 4 pontos percentuais entre treino e validação nas épocas finais é esperada e compatível com o comportamento típico de redes treinadas do zero em datasets de porte médio (DENG et al., 2014).



**Figura 5:** Evolução do erro (loss) — CNN V7 CBAM (treino vs. validação).

**Fonte:** Autoria própria (2026).

A curva de loss, apresentada na Figura 5, complementa essa análise. Tanto a loss de treino quanto a de validação apresentaram decaimento acentuado nas primeiras épocas saindo de aproximadamente 0.53 e 0.41 respectivamente, seguido de uma convergência suave e estável. A loss de validação manteve-se consistentemente abaixo da loss de treino ao longo de praticamente todo o treinamento, comportamento diretamente atribuível à aplicação de augmentation online exclusivamente no conjunto de treino, que torna a tarefa de treino artificialmente mais difícil e à ausência de augmentation no conjunto de validação. Dois picos isolados de loss de validação são observáveis em torno das épocas 25 e 30, os quais correspondem a reduções automáticas da taxa de aprendizado pelo callback

ReduceLROnPlateau, que momentaneamente desestabilizaram a loss antes de induzir ajustes mais finos nos pesos. Após cada redução, a curva retomou a tendência decrescente, validando a eficácia do esquema de redução adaptativa da taxa de aprendizado configurado com fator 0.2 e paciência de 4 épocas.

## 4.2 - Avaliação sem TTA

A avaliação inicial do modelo foi realizada sobre o conjunto de **teste** (avaliação final) de 1.448 amostras sem aplicação de Test-Time Augmentation, obtendo acurácia global de 92%, conforme apresentado na Tabela 2.

**Tabela 2:** Relatório de classificação sem TTA (conjunto de teste, n=1.448).

| Classe                 | Precision   | Recall      | F1-score    | Support     |
|------------------------|-------------|-------------|-------------|-------------|
| bacterial_blight       | 0.84        | 0.96        | 0.90        | 73          |
| cercospora_leaf_blight | 0.93        | 0.89        | 0.91        | 240         |
| downey_mildew          | 0.92        | 1.00        | 0.96        | 98          |
| frogeye                | 0.86        | 0.98        | 0.92        | 231         |
| healthy                | 0.92        | 0.90        | 0.91        | 245         |
| potassium_deficiency   | 1.00        | 0.93        | 0.96        | 155         |
| soybean_rust           | 0.95        | 0.87        | 0.91        | 244         |
| target_spot            | 0.98        | 0.95        | 0.97        | 162         |
| <b>macro avg</b>       | <b>0.92</b> | <b>0.93</b> | <b>0.93</b> | <b>1448</b> |
| <b>weighted avg</b>    | <b>0.93</b> | <b>0.92</b> | <b>0.92</b> | <b>1448</b> |

**Fonte:** Autoria própria (2026).

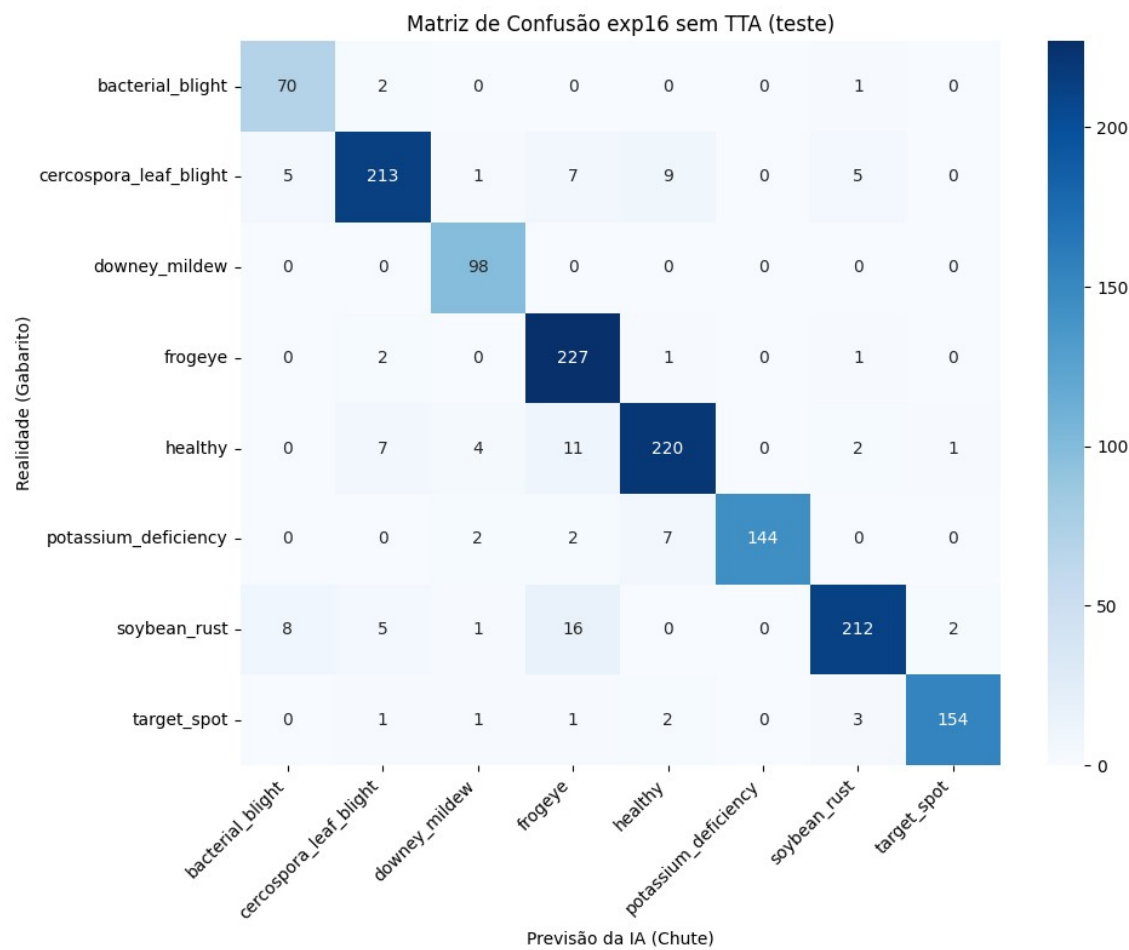


Figura 6: Matriz de Confusão sem TTA — conjunto de teste.

Fonte: Autoria própria (2026).

A análise da matriz de confusão e do relatório de classificação, demonstrada na Figura 6, revelou desempenho sólido e homogêneo na maioria das classes. A classe downey\_mildew obteve recall perfeito de 1.00, classificando corretamente todas as 98 amostras do conjunto de teste, com F1-score de 0.96 o segundo maior valor individual observado nessa avaliação. As classes potassium\_deficiency e target\_spot também apresentaram resultados expressivos, com F1-score de 0.96 e 0.97 respectivamente, refletindo a distintividade visual de seus padrões sintomáticos: o amarelamento internerval característico da deficiência de potássio e as lesões circulares concêntricas do target spot apresentam baixa sobreposição visual com as demais categorias do dataset.

As classes bacterial\_blight e frogeye registraram os valores mais baixos de precision, 0.84 e 0.86 respectivamente, indicando presença de falsos positivos nessas categorias. A análise da matriz de confusão revela que o principal vetor de confusão da arquitetura sem TTA foi a classe soybean\_rust, que gerou 16 classificações incorretas como frogeye e 8 como



bacterial\_blight, resultando em recall de 0.87 para soybean\_rust, o mais baixo entre todas as classes. Esse padrão de confusão triangular entre soybean\_rust, frogeye e bacterial\_blight reflete a sobreposição visual real entre essas três patologias, especialmente em imagens onde as lesões se encontram em estágios iniciais ou com densidade reduzida, e será discutido em detalhe na seção 4.4. A classe cercospora\_leaf\_blight também apresentou dispersão relevante de erros, com 9 amostras classificadas como healthy e 7 como frogeye, padrão igualmente abordado na análise qualitativa da seção 4.4.

### 4.3 - Avaliação com TTA

A avaliação com Test-Time Augmentation foi conduzida aplicando cinco variantes de transformação sobre cada imagem do conjunto de teste de 1.448 amostras, imagem original, flip horizontal, flip vertical, rotação de 90° e rotação de 270°, e calculando a média das distribuições de probabilidade resultantes como predição final. Esse procedimento elevou a acurácia global de 92% para 94%, representando um ganho absoluto de 2 pontos percentuais sem qualquer alteração na arquitetura ou nos pesos do modelo. A Tabela 3 apresenta as métricas obtidas com TTA.

**Tabela 3:** Relatório de classificação com TTA (conjunto de teste, n=1.448).

| Classe                 | Precision   | Recall      | F1-score    | Support     |
|------------------------|-------------|-------------|-------------|-------------|
| bacterial_blight       | 0.86        | 0.95        | 0.90        | 73          |
| cercospora_leaf_blight | 0.93        | 0.91        | 0.92        | 240         |
| downey_mildew          | 0.94        | 1.00        | 0.97        | 98          |
| frogeye                | 0.88        | 0.99        | 0.93        | 231         |
| healthy                | 0.93        | 0.93        | 0.93        | 245         |
| potassium_deficiency   | 1.00        | 0.95        | 0.98        | 155         |
| soybean_rust           | 0.96        | 0.86        | 0.90        | 244         |
| target_spot            | 0.99        | 0.98        | 0.98        | 162         |
| <b>macro avg</b>       | <b>0.94</b> | <b>0.95</b> | <b>0.94</b> | <b>1448</b> |
| <b>weighted avg</b>    | <b>0.94</b> | <b>0.94</b> | <b>0.94</b> | <b>1448</b> |

Fonte: Autoria própria (2026).

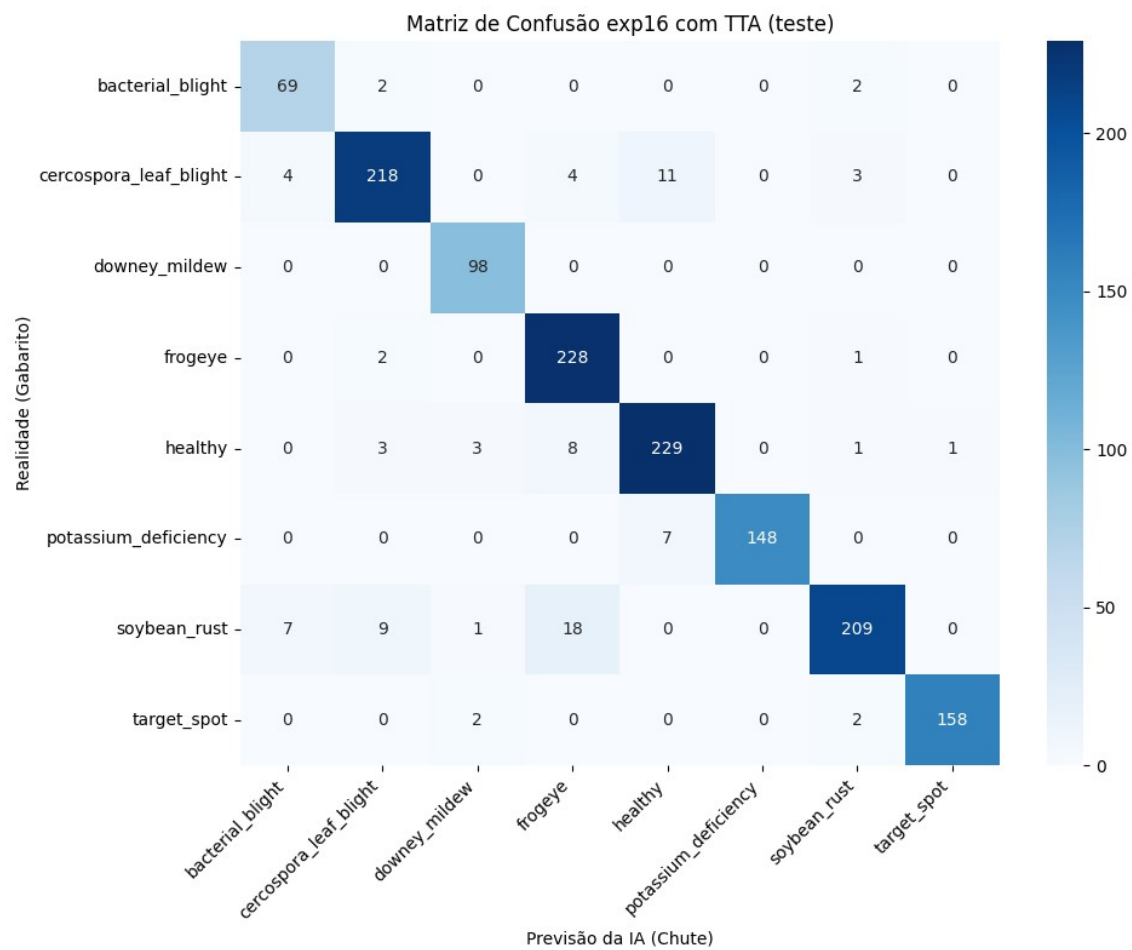


Figura 7: Matriz de Confusão com TTA — conjunto de teste.

Fonte: Autoria própria (2026).

Como pode ser observado na Figura 7, a comparação direta entre os dados obtidos e as métricas das Tabelas 2 e 3 evidencia que o TTA produziu ganhos consistentes na maioria das classes. A classe *downey\_mildew* manteve *recall* perfeito de 1.00 e elevou sua *precision* de 0.92 para 0.94, atingindo *F1-score* de 0.97 — o segundo maior valor individual da avaliação com TTA. A classe *frogeye* registrou o ganho de *recall* mais expressivo, passando de 0.98 para 0.99, com uma redução de erros que se manifesta de forma explícita na distribuição da Figura 7. A classe *healthy* apresentou melhora equilibrada, com *precision* subindo de 0.92 para 0.93 e *recall* de 0.90 para 0.93, refletindo que a média de múltiplas perspectivas transformadas ajudou a disambiguar casos limítrofes dessa classe. A classe *potassium\_deficiency* atingiu *precision* perfeita de 1.00 com TTA, e a categoria *target\_spot* alcançou *F1-score* de 0.98, consolidando o maior valor individual observado em toda a avaliação.

O único caso em que o TTA não produziu melhora expressiva foi a classe *soybean\_rust*, cujo *recall* diminuiu ligeiramente de 0.87 para 0.86, e cujo número de confusões com *frogeye* aumentou ligeiramente de 16 para 18 amostras, conforme mapeado na Figura 7.

Esse comportamento indica que os erros residuais de *soybean\_rust* representam casos genuinamente ambíguos nos quais as múltiplas orientações geométricas não fornecem informação discriminativa adicional suficiente para reverter a predição incorreta, possivelmente porque a sobreposição visual entre *soybean\_rust* e *frogeye* é intrínseca às próprias amostras e não decorrente de variações de orientação ou enquadramento. Esse padrão de confusão persistente é discutido em detalhe na seção 4.4.

O *macro F1-score* de 0.94 e o *weighted F1-score* de 0.94 confirmam que o modelo apresentou desempenho equilibrado e robusto entre as oito classes do ASDID.

## 4.4 - Análise dos Principais Erros de Classificação

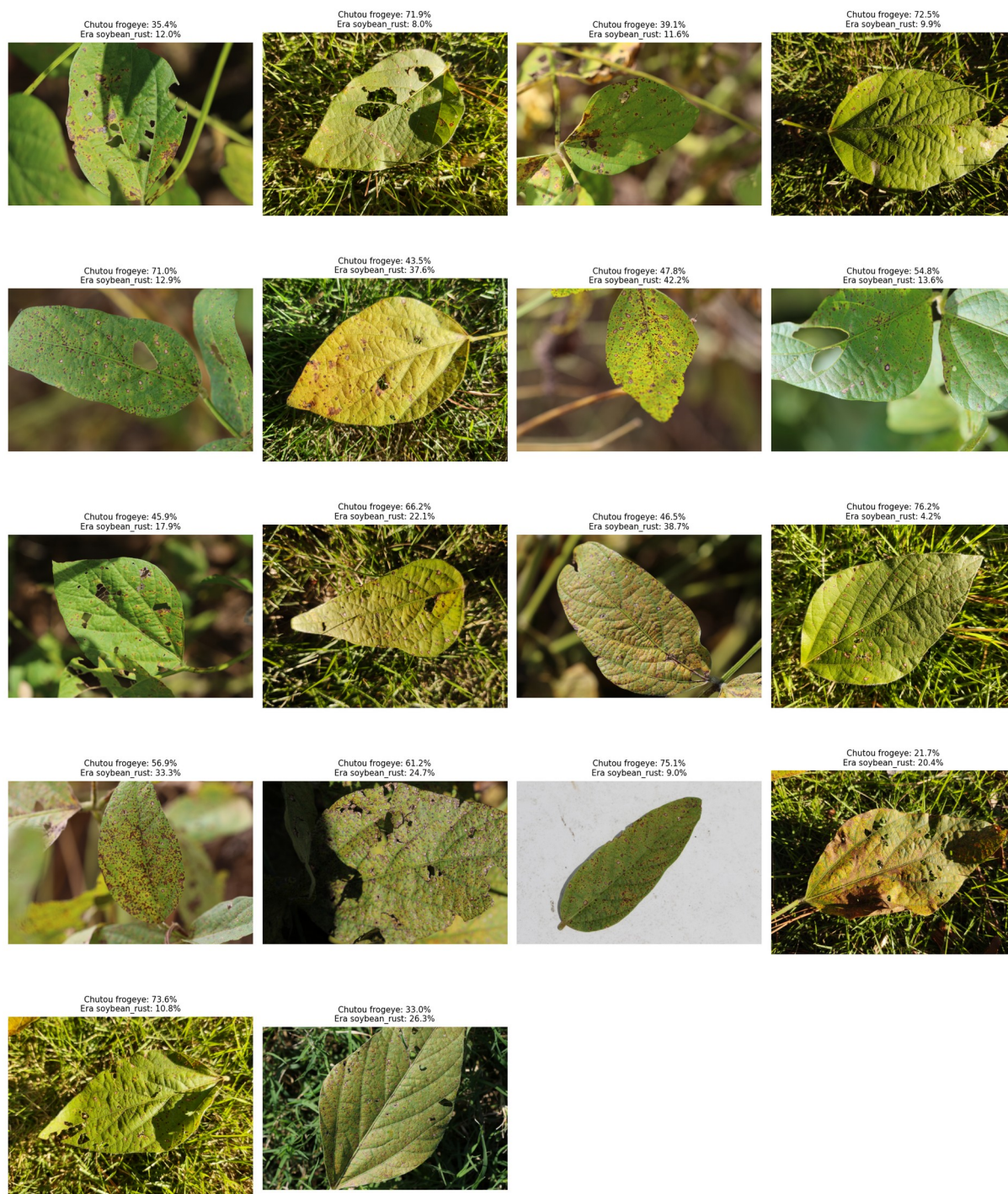
A análise qualitativa dos erros residuais do modelo foi conduzida sobre a configuração de teste com TTA, que representa a avaliação final do sistema. Com base na matriz de confusão, foram identificados seis padrões de confusão, dos quais os dois mais frequentes, *soybean\_rust* classificado como *frogeye* (n=18) e *cercospora\_leaf\_blight* classificado como *healthy* (n=11), foram investigados por meio de painéis visuais das amostras mal classificadas. Os demais padrões identificados, *soybean\_rust* confundido com *cercospora\_leaf\_blight* (n=9), *soybean\_rust* confundido com *bacterial\_blight* (n=7), *potassium\_deficiency* confundida com *healthy* (n=7) e *healthy* confundida com *frogeye* (n=8), são discutidos textualmente com base nas características visuais observadas nas matrizes de confusão.

### 4.4.1 - Confusão entre *soybean\_rust* e *frogeye*

A confusão mais frequente observada no conjunto de teste com TTA foi a classificação de amostras de *soybean\_rust* como *frogeye*, totalizando 18 ocorrências. A análise visual do painel apresentado na Figura 8 revelou um padrão consistente e biologicamente interpretável: a grande maioria das amostras mal classificadas apresentava pústulas de ferrugem em densidade baixa a moderada, distribuídas esparsamente sobre a superfície foliar. Nessas condições, as pústulas isoladas, pequenos pontos avermelhados a acastanhados

sobre fundo verde, assumem aparência visual similar às lesões iniciais do *frogeye leaf spot*, que também se manifesta como pontuações circulares discretas sobre o limbo foliar verde.

Ferrugem confundida com Frogeye (n=18)



**Figura 8:** Painel de erros — soybean\_rust classificado como frogeye (n=18).

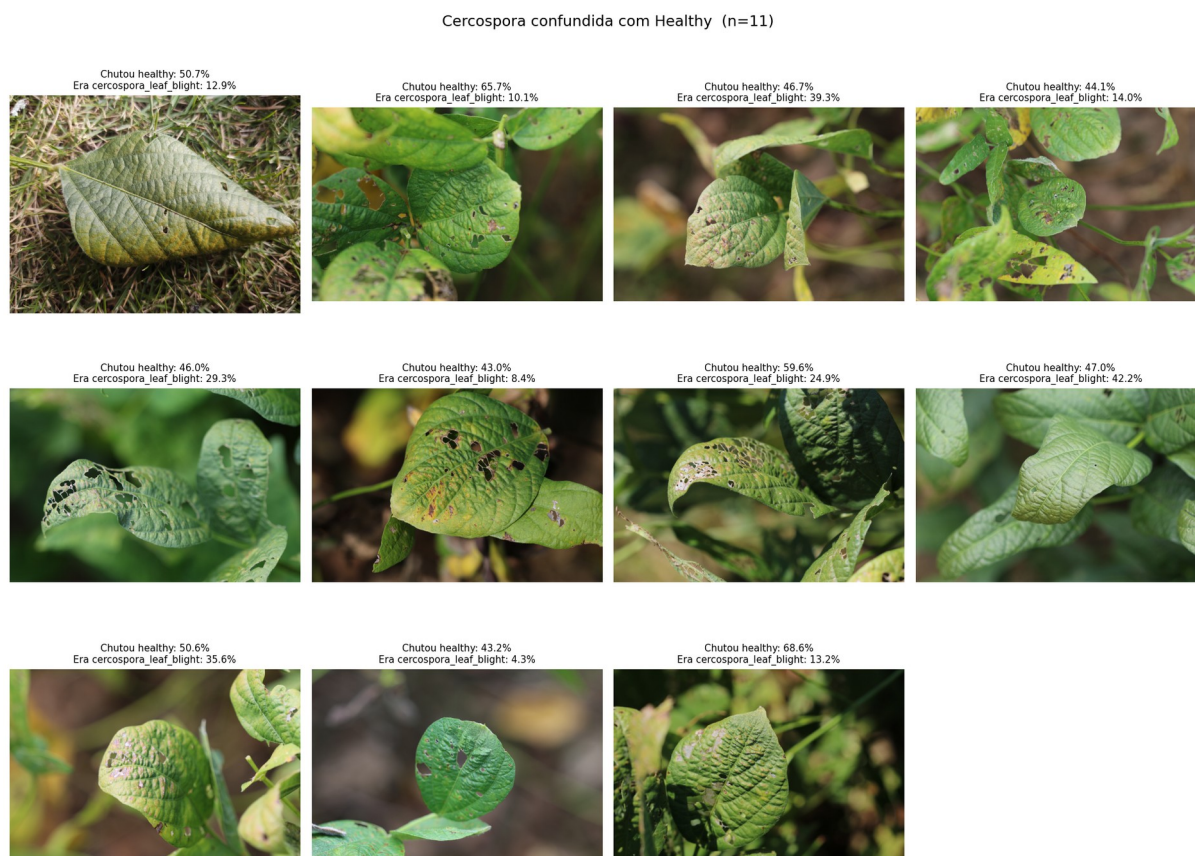
**Fonte:** Autoria própria (2026).

As probabilidades preditas para cada imagem na Figura 8 confirmam a ambiguidade: em 14 das 18 amostras, a confiança do modelo para *frogeye* ficou entre 33% e 76%, enquanto a probabilidade real de *soybean\_rust* oscilou entre 4% e 42%, indicando que a rede reconheceu a incerteza mas não foi capaz de resolver o conflito em favor da classe correta. Casos como o de confiança 21.7% para *frogeye* e 20.4% para *soybean\_rust*, também ilustrados na Figura 8, exemplificam amostras genuinamente ambíguas, onde mesmo a inspeção visual humana poderia resultar em divergência diagnóstica. Esse padrão de confusão é documentado na literatura como um dos pares mais desafiadores em classificação automática de doenças foliares em soja, dada a sobreposição fenomenológica entre as manifestações iniciais de ambas as patologias (BARBEDO, 2013).

#### **4.4.2 -Confusão entre cercospora\_leaf\_blight e healthy**

O segundo padrão de confusão mais relevante foi a classificação de amostras de *cercospora\_leaf\_blight* como *healthy*, com 11 ocorrências. A análise visual do painel apresentado na Figura 9 identificou dois subtipos distintos de erro. O primeiro subtipo, mais frequente, corresponde a imagens onde a infecção por *cercospora* encontrava-se em estágio inicial ou muito localizado: manchas de coloração púrpura-acastanhada restritas a pequenas regiões da folha, com a maior parte do limbo foliar apresentando coloração verde uniforme e aspecto saudável. Nesses casos, o mecanismo de atenção espacial do CBAM não foi suficiente para sobrepor a evidência da região lesionada à dominância visual da área sã, resultando em predições para *healthy* com confiança entre 43% e 68%, conforme detalhado nos rótulos de predição da Figura 9.





**Figura 9:** Painel de erros — cercospora\_leaf\_blight classificado como healthy (n=11).

**Fonte:** Autoria própria (2026).

O segundo subtipo corresponde a imagens com fatores externos que mascararam os sinais patológicos: curvatura acentuada da folha, fundo complexo com gramíneas ou outras plantas ao redor, e variações severas de iluminação lateral que reduziram o contraste entre as lesões e o tecido são adjacente, conforme pode ser observado diretamente na folha sobre grama e nas amostras com fundo escuro que integram a Figura 9. Em ambos os subtipos, as probabilidades para cercospora\_leaf\_blight registradas como segunda opção ficaram entre 4% e 42%, confirmando que o modelo detectou parcialmente a presença da patologia mas não com confiança suficiente para sobrepor a predição de healthy. Esse comportamento é consistente com a natureza da cercospora em soja, cuja sintomatologia inicial pode ser indistinguível de danos mecânicos, senescência natural de tecido foliar ou simples imperfeições foliares sem caráter patológico (HENNING et al., 2014).

#### 4.4.3 -Demais padrões de confusão

Os padrões secundários identificados no conjunto de teste com TTA seguem lógica visual similar. A confusão de soybean\_rust com cercospora\_leaf\_blight (n=9) ocorre em imagens onde as pústulas de ferrugem, em estágio avançado, produzem manchas de coloração acastanhada difusa que se assemelha às lesões de cercospora. A confusão de soybean\_rust com bacterial\_blight (n=7) reflete o mesmo mecanismo já documentado: lesões de ferrugem de baixa densidade cujas pontuações isoladas mimetizam o padrão de manchas angulares do bacterial\_blight. A confusão de potassium\_deficiency com healthy (n=7) ocorre predominantemente em imagens com sintomas iniciais de amarelamento internerval ainda não expressivos o suficiente para diferenciar-se visualmente de variações normais de coloração foliar. Por fim, a confusão de healthy com frog-eye (n=8) indica que algumas amostras da classe healthy apresentavam imperfeições, furos ou danos mecânicos que o modelo interpretou como lesões características do frog-eye leaf spot.

Em conjunto, a análise dos erros residuais demonstra que os casos mal classificados pelo modelo não representam falhas sistemáticas da arquitetura, mas sim amostras genuinamente limítrofes onde a ambiguidade visual é inerente à própria natureza das patologias em estágios iniciais ou sob condições adversas de captura. Esse tipo de erro de fronteira é esperado e documentado na literatura de classificação automática de doenças foliares, mesmo em modelos de alta performance (BARBEDO, 2019; ZHANG et al., 2025).

## 5 - Conclusão

Este capítulo apresenta a conclusão final deste trabalho por meio de duas seções. A seção 5.1 apresenta as considerações finais. A seção 5.2 aponta direcionamentos de trabalhos futuros.

### 5.1 - Considerações Finais

O presente trabalho propôs e desenvolveu um sistema de triagem de doenças foliares em soja, com foco no rigor técnico do modelo de aprendizado profundo e na viabilidade de um serviço de inferência em backend. O objetivo principal foi alcançado: a arquitetura CNN V7 CBAM, projetada e treinada do zero sobre o dataset ASDID, demonstrou capacidade de classificar as oito classes patológicas do dataset com desempenho consolidado, atingindo 92% de acurácia global no conjunto de teste sem augmentation de inferência e macro F1-score de 0.94 com a aplicação de Test-Time Augmentation sobre cinco variantes geométricas. Esses resultados validam a hipótese central do trabalho: que uma arquitetura convolucional customizada, dispensando modelos pré-treinados externos, é suficientemente capaz de aprender representações discriminativas de lesões foliares a partir de um dataset de porte médio, desde que dotada de mecanismos adequados de regularização, atenção e função de perda adaptativa.

A decisão arquitetural de integrar o Convolutional Block Attention Module (CBAM) após cada bloco convolucional mostrou-se tecnicamente justificada pelos resultados obtidos. O mecanismo de atenção de canal e espacial direcionou o aprendizado da rede para as regiões sintomáticas das folhas, reduzindo a influência de fundo complexo e tecido foliar sadio dominante problema crítico em datasets capturados em condições reais de campo. A adoção da Focal Loss customizada com vetores de alpha por classe permitiu tratar o desequilíbrio intrínseco do ASDID de forma mais granular do que técnicas convencionais de ponderação por classe, com ganhos observáveis especialmente nas classes minority como bacterial\_blight e frog-eye leaf spot. O pipeline de dados construído nativamente com TensorFlow integrando tf.data, tf.io, tf.image e camadas Keras de augmentation com paralelismo AUTOTUNE garantiu eficiência computacional e reprodutibilidade integral do processo de treinamento, que convergiu na época 48 com tempo total de aproximadamente 1.160 minutos.



O modelo treinado foi encapsulado em uma API de inferência de backend desenvolvida com FastAPI e servida pelo servidor ASGI Uvicorn, disponível em <http://localhost:8000>. A API expõe o endpoint POST /predict, que recebe uma imagem foliar, aplica o pipeline de pré-processamento e executa a inferência com Test-Time Augmentation de cinco variantes, retornando em JSON a classe prevista em português, o nível de confiança e as probabilidades das oito classes. As camadas customizadas do CBAM foram registradas como objetos customizados no momento do carregamento do modelo, garantindo a deserialização correta da arquitetura. O frontend desenvolvido em HTML, CSS e JavaScript consome essa API diretamente, exibindo os resultados com barras de probabilidade animadas e classificação visual por cor, verde para folha saudável e vermelho para patologia detectada, constituindo uma demonstração funcional e completa do sistema de triagem proposto.

O conjunto dos resultados obtidos posiciona o trabalho como uma contribuição técnica concreta à interseção entre Sistemas de Informação e agricultura de precisão, demonstrando que é viável construir, treinar e disponibilizar um classificador de doenças foliares em soja com métricas competitivas, baixo custo arquitetural e arquitetura de software orientada à integração sem dependência de infraestrutura de nuvem proprietária ou modelos de grande porte.

## 5.2 - Limitações e Trabalhos Futuros

Apesar dos resultados expressivos, o trabalho apresenta limitações que devem ser reconhecidas. A principal delas reside na origem exclusiva dos dados: todas as imagens utilizadas no treinamento, validação e teste pertencem ao Auburn Soybean Disease Image Dataset (ASDID), base coletada em condições específicas de campo e laboratório nos Estados Unidos. A ausência de imagens capturadas em lavouras brasileiras impede afirmar que o modelo generalizará com o mesmo desempenho para cultivares locais submetidas às variações genéticas, climáticas e de manejo características das regiões produtoras do Brasil como o Cerrado e o Sul do país. A adaptação do modelo a esse contexto exigiria a coleta e anotação de um dataset complementar com imagens nacionais, o que constitui uma limitação prática relevante para a aplicação imediata da solução em campo.

Uma segunda limitação diz respeito à ambiguidade visual intrínseca entre determinados pares de classes. A análise dos erros residuais evidenciou que as confusões mais

frequentes soybean\_rust com frogeye (n=18), cercospora\_leaf\_blight com healthy (n=11) e soybean\_rust com bacterial\_blight (n=7), não decorrem de deficiências arquiteturais do modelo, mas da sobreposição fenomenológica real entre essas patologias em estágios iniciais de infecção. Lesões de ferrugem em baixa densidade de pústulas são visualmente quase indistinguíveis das manchas circulares do frogeye leaf spot; sintomas iniciais de cercospora, quando restritos a pequenas regiões da folha, podem ser confundidos com imperfeições naturais de tecido saudável. Esse problema é inerente à tarefa e afetaria qualquer modelo matemático submetido às mesmas amostras, sendo documentado na literatura como um dos desafios centrais da classificação automática de doenças foliares (BARBEDO, 2013). A ampliação do dataset com imagens de diferentes estágios fenológicos das patologias, especialmente de manifestações iniciais, representa o caminho mais direto para mitigar esses erros de fronteira.

Com base nas limitações identificadas e nos resultados obtidos, três direcionamentos principais são propostos para trabalhos futuros. O primeiro consiste no desenvolvimento de uma interface gráfica web ou mobile para consumo direto da API de inferência por produtores rurais e técnicos agrícolas em campo, tornando a solução acessível a usuários sem conhecimento técnico em programação e viabilizando o uso prático do sistema de triagem desenvolvido neste trabalho. O segundo direcionamento consiste na ampliação do dataset com imagens coletadas em lavouras brasileiras, capturadas em diferentes regiões produtoras, estágios fenológicos e condições de iluminação, com o objetivo de adaptar e refinar a rede às particularidades das cultivares e do ambiente agrícola nacional etapa indispensável para que a solução possa ser considerada robusta para uso em campo no Brasil. O terceiro direcionamento propõe a investigação do embarque do modelo de inferência em dispositivos de baixo consumo energético, como drones agrícolas ou dispositivos IoT instalados em campo, viabilizando o monitoramento autônomo e contínuo de lavouras sem dependência de conectividade constante com servidores remotos aplicação com alto potencial de impacto para a agricultura de precisão em regiões com infraestrutura de conectividade limitada.

Como recomendações decorrentes desses direcionamentos, sugere-se que trabalhos futuros priorizem a validação do modelo em parceria com cooperativas agrícolas, casas de assistência técnica ou unidades experimentais da Embrapa, de modo a obter retroalimentação real de campo antes de qualquer disponibilização ampla da ferramenta. Recomenda-se

também que a adoção do sistema em ambiente produtivo seja sempre acompanhada da orientação explícita de que a ferramenta possui caráter de triagem e apoio à decisão, não substituindo o diagnóstico fitopatológico realizado por profissional habilitado, sobretudo nos casos em que o nível de confiança retornado pelo modelo for baixo. Por fim, recomenda-se a adoção de um processo de monitoramento contínuo do desempenho do modelo após eventual implantação, de modo a identificar precocemente eventual degradação das métricas diante de novas condições de captura, cultivares ou patologias emergentes, garantindo a confiabilidade da solução ao longo do tempo.

## Referências Bibliográficas

- ABADI, M. et al. TensorFlow: a system for large-scale machine learning. In: USENIX SYMPOSIUM ON OPERATING SYSTEMS DESIGN AND IMPLEMENTATION, 12., 2016, Savannah. *Proceedings...* Savannah: USENIX Association, 2016. p. 265-283.
- BARBEDO, J. G. A. Digital image processing techniques for detecting, quantifying and classifying plant diseases. *SpringerPlus*, [S. l.], v. 2, n. 660, 2013.
- BARBEDO, J. G. A. A review on the main challenges in automatic plant disease identification based on visible range images. *Biosystems Engineering*, [S. l.], v. 183, p. 83-99, 2019.
- BEVERS, N.; SIKORA, E. J.; HARDY, N. B. Soybean disease identification using original field images and transfer learning with convolutional neural networks. *Computers and Electronics in Agriculture*, [S. l.], v. 203, 107449, 2022.
- BROSNAN, T.; SUN, D. W. Improving quality inspection of food products by computer vision: a review. *Journal of Food Engineering*, [S. l.], v. 61, n. 1, p. 3-16, 2004.
- CARVALHO, R. As aplicações da inteligência artificial na agricultura. *TOTVS Blog*, 2024. Disponível em: <https://www.totvs.com/blog/agronegocio/inteligencia-artificial-na-agricultura/>. Acesso em: 14 set. 2025.
- CNA; CEPEA. PIB do agronegócio registra crescimento de 6,49% no primeiro trimestre de 2025. Brasília: CNA, 2025. Disponível em: <https://www.cnabrazil.org.br/publicacoes/pib-do-agronegocio-registra-crescimento-de-6-49-no-primeiro-trimestre-de-2025>. Acesso em: 22 ago. 2025.
- CONAB. Acompanhamento da safra brasileira de grãos: safra 2024/25. Brasília: Conab, 2024. Disponível em: <https://www.conab.gov.br/info-agro/safras/graos>. Acesso em: 12 jan. 2026.
- DENG, L.; YU, D. Deep Learning: Methods and Applications. *Foundations and Trends in Signal Processing*, [S. l.], v. 7, n. 3-4, p. 197-387, 2014.
- FIELDVIEW. Principais doenças da soja: saiba como identificá-las. 2020. Disponível em: <https://blog.climatefieldview.com.br/principais-doencas-da-soja>. Acesso em: 3 out. 2025.
- FUKUSHIMA, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, [S. l.], v. 36, n. 4, p. 193-202, 1980.
- GEOAGRI. Agricultura de Precisão. [S. d.]. Disponível em: <https://www.geoagri.com.br>. Acesso em: 18 set. 2025.
- GODOY, C. V. et al. Eficiência de fungicidas para o controle da ferrugem-asiática da soja, *Phakopsora pachyrhizi*, na safra 2022/2023: resultados sumarizados dos ensaios

cooperativos. Londrina: Embrapa Soja, 2023. (Circular Técnica, 195). Disponível em: <https://zenodo.org/records/7304859>. Acesso em: 7 nov. 2025.

GRYFO. O que é Visão Computacional? 2021. Disponível em: <https://gryfo.com.br>. Acesso em: 25 set. 2025.

HAN, S. et al. A review of computer vision technologies for plant phenotyping. *Computers and Electronics in Agriculture*, [S. l.], v. 98, p. 199-216, 2013.

HENNING, A. A. et al. *Manual de identificação de doenças de soja*. 5. ed. Londrina: Embrapa Soja, 2014. (Documentos, 256). Disponível em: <https://www.alice.cnptia.embrapa.br/alice/bitstream/doc/1128404/1/p.-227-264-de-SP-17-2-020-online.pdf>. Acesso em: 12 out. 2025.

KINGMA, D. P.; BA, J. Adam: a method for stochastic optimization. arXiv preprint, arXiv:1412.6980, 2015. Disponível em: <https://arxiv.org/abs/1412.6980>. Acesso em: 19 jan. 2026.

LI, X. et al. Soybean Leaf Disease Recognition Methods Based on Hyperparameter Transfer and Progressive Fine-Tuning of Large Models. *Agronomy*, [S. l.], v. 16, 218, 2026. Disponível em: <https://doi.org/10.3390/agronomy16020218>. Acesso em: 15 jan. 2026.

LIN, T. et al. Focal Loss for Dense Object Detection. In: IEEE INTERNATIONAL CONFERENCE ON COMPUTER VISION, 2017, Veneza. *Proceedings...* Veneza: IEEE, 2017. p. 2980-2988. Disponível em: <https://arxiv.org/abs/1708.02002>. Acesso em: 19 jan. 2026.

REUTERS. Brazil's soybean exports to hit record as US out of market, Chinese demand strong. [S. l.]: Reuters, 2025. Disponível em: <https://www.reuters.com/sustainability/climate-energy/brazils-soybean-exports-hit-record-us-out-market-chinese-demand-strong-2025-10-08/>. Acesso em: 28 nov. 2025.

SEIXAS, C. D. S. et al. (Ed.). *Tecnologias de produção de soja*. Londrina: Embrapa Soja, 2020. (Sistemas de Produção, 17). Disponível em: <https://www.infoteca.cnptia.embrapa.br/infoteca/bitstream/doc/1123928/1/SP-17-2020-online-1.pdf>. Acesso em: 5 out. 2025.

WOO, S. et al. CBAM: Convolutional Block Attention Module. In: EUROPEAN CONFERENCE ON COMPUTER VISION, 15., 2018, Munich. *Proceedings...* Munich: Springer, 2018. p. 3-19.

YU, M. et al. A recognition method of soybean leaf diseases based on an improved deep learning model. *Frontiers in Plant Science*, [S. l.], v. 13, 878834, 2022. Disponível em: <https://doi.org/10.3389/fpls.2022.878834>. Acesso em: 8 mar. 2026.

ZHANG, X. et al. Soybean leaf disease identification based on multi-scale feature extraction and fusion. *Frontiers in Plant Science*, [S. l.], v. 16, 1655564, 2025. Disponível em: <https://doi.org/10.3389/fpls.2025.1655564>.

Acesso em: 15 jan. 2026.